



O'REILLY®

AI Memory Management in Agentic Systems

By Richmond Alake

Empowering AI agents with robust memory

Meet your Instructor

Richmond Alake

- **Director of AI Developer Experience at Oracle:** Leads AI developer experience and outreach for Oracle AI Database, focusing on open-source framework integrations, AI ecosystem, and developer enablement.
- **AI Engineer & Computer Vision Specialist:** Holds an MSc in Computer Vision, Machine Learning and Robotics from the University of Surrey, with software development and ML engineering experience since around 2016. Former ML Architect at Slalom Build and AI/ML Developer Advocate at MongoDB.
- **Technical Writer & Educator:** Authored 200+ technical articles with over 1 million views across Medium, MongoDB Developer Hub, and the NVIDIA Developer Blog. Co-instructor of a DeepLearning.AI short course alongside Andrew Ng's team and lead instructor for an O'Reilly live training bootcamp.
- **Memory Engineering Pioneer:** Coined "Memory Engineering" as a named AI discipline and built a sustained public identity around it through the #100DaysOfAgentMemory series.
- **AI Stack Engineer:** Creator of MemoRizz (open-source agent memory library) and the Oracle AI Agent Memory Package.





Course Schedule: Module 1 of 3

AI Agent Memory · from definitions to disciplines

MODULE 1 · AI AGENT MEMORY *the complete conceptual foundation: every term the rest of the day uses, defined, drawn and defended*

01	Defining agent memory	what an AI agent is (model + harness), why memory: the reset problem, and the four Rs: retain, recall, reuse, refine
02	The Unified Memory Core	one taxonomy read four ways: three tiers, six functions, ten implementations, split by duration first
03	Short-term memory	working memory in the window and as session storage, a live case study, and the semantic cache with its ~40x economics
04	Long-term memory	skills, stories and facts; procedural memory in depth: workflow memory, the toolbox, and the skills registry
05	Management & the lifecycle	the six operations, the memory engineering lifecycle with all four Rs on one diagram, and what memory buys: reliable, believable, capable
06	Architecture & the layers	the reference architecture, the Agent Stack, and the memory, semantic, ontology and context layers with their dictionary entries
07	The three disciplines	prompt, context and memory engineering: their techniques, the context flywheel, the eleven-stage lifecycle, and the memory engineer's role
08	Industry update · Aug 2026	five labs side by side, the dreaming convergence, the two-year timeline, and Computer History: memory as the moat



Course Schedule: Module 2 of 3

AI Memory Techniques · the layer stack, opened

MODULE 2 · AI MEMORY TECHNIQUES *six layers, technique by technique: definitions, the papers that earned them, and reference architectures*

01	How form factors evolved	LLM chatbots to RAG to agents to agentic systems: what each jump required, and why the retrieval layer never left
02	The six-layer map	prompt, retrieval, context, memory, semantic, ontology: one stack, with ownership marked at every level
03	The prompt layer	chain of thought, ReAct and few-shot examples, each shown where it lives: in the window, and in the tokens it spends
04	The retrieval layer	lexical search, embedding models, distance functions, chunking, HNSW, the hybrid search stack, reranking and quantization
05	The context layer	just-in-time retrieval with placeholder references, offloading, and reduction: summarise the prose, compact the structure
06	The memory layer	consolidation and forgetting (the Generative Agents score), augmentation at ingestion, and the files-versus-database substrate
07	Semantic & ontology layers	governed definitions, comments and usage harvesting; then what exists and how it relates: entities, relations, resolution, constraints

Course Schedule: Module 3 of 3

Memory Engineering In Action · concepts meet code



MODULE 3 · MEMORY ENGINEERING IN ACTION *the discipline practised: hands-on implementation, the tooling landscape, and where to go next*

- | | | |
|----|--|--|
| 01 | The memory engineer's role | six ownerships in one job description: the schema, the lifecycle, recall, the evolution, the evidence and the trust |
| 02 | The twelve techniques, applied | from memory unit modelling and extraction through consolidation and dreaming to evaluation, security and continual learning |
| 03 | Hands-on with MemoRizz | the open-source teaching library: memory provider and MemAgent classes, and each memory type implemented, retrieved and observed |
| 04 | Frameworks & managed services | LangMem and LangGraph stores, MemGPT patterns, Mem0's graph memory, and Vertex AI Memory Bank as the managed path |
| 05 | Reading list & next steps | the papers, surveys and courses that continue after the room empties, plus the closing poll: where will you apply this first? |

Module 1: AI Agent Memory





Defining AI Agent Memory

Why Memory

Overview of AI Memory

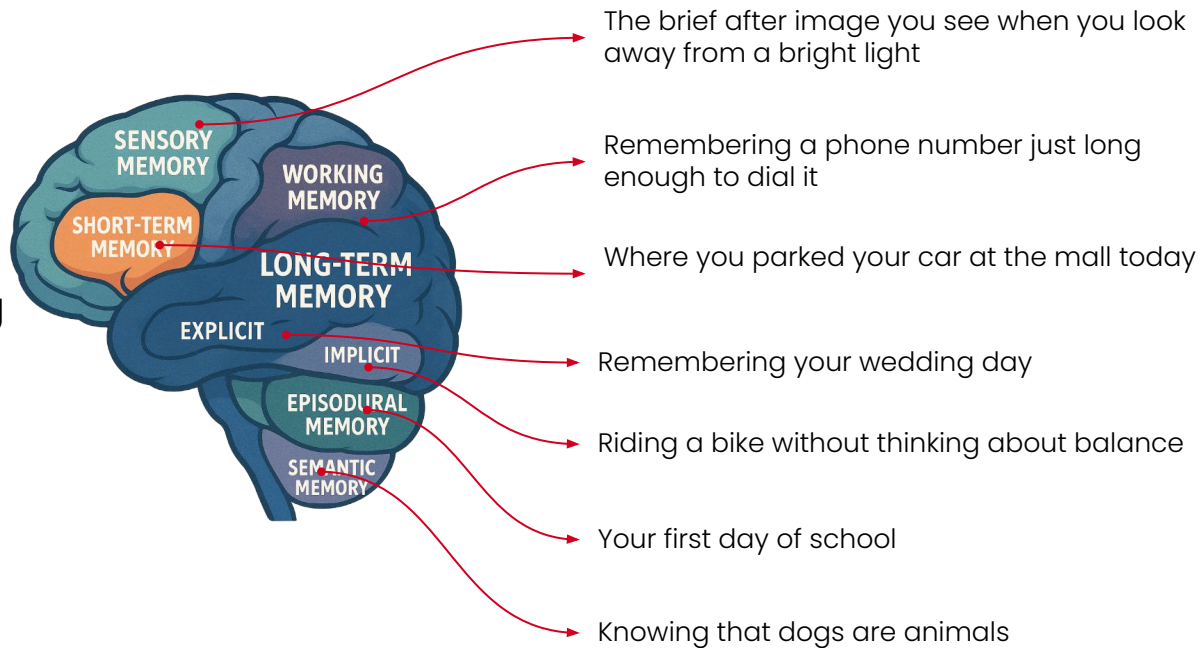
Artificial Intelligence is the scientific endeavor to create a computational form of **intelligence distinct from organic intelligence**, one that convincingly **mimics human cognitive abilities**.

Artificial General Intelligence (AGI) refers to artificial intelligence that **surpasses human performance across most tasks** traditionally considered solvable by human intelligence.

How Memory Works in Human (Loosely)

Overview of AI Memory

Humans have a good ability to **retain, recall** and **reuse** information over **short and long period** of time.



What Is An AI Agent?

Overview of AI Memory

AI agent /ˌeɪˌaɪ ˈeɪ.dʒənt/ noun

from the Latin *agere*: to do, to drive, to act

- 1 An AI agent is a computational entity that can **perceive its environment**, **reason** using a large language model, **retain and recall information** through memory, and take **autonomous action** through tool use, operating in a **continuous loop** to achieve goals without constant human direction.
- 2 *(in this course)* A **model** wrapped in a **harness**: raw capability channelled into reliable, autonomous action.

agent = model + harness

PERCEIVES

the environment

REASONS

via an LLM

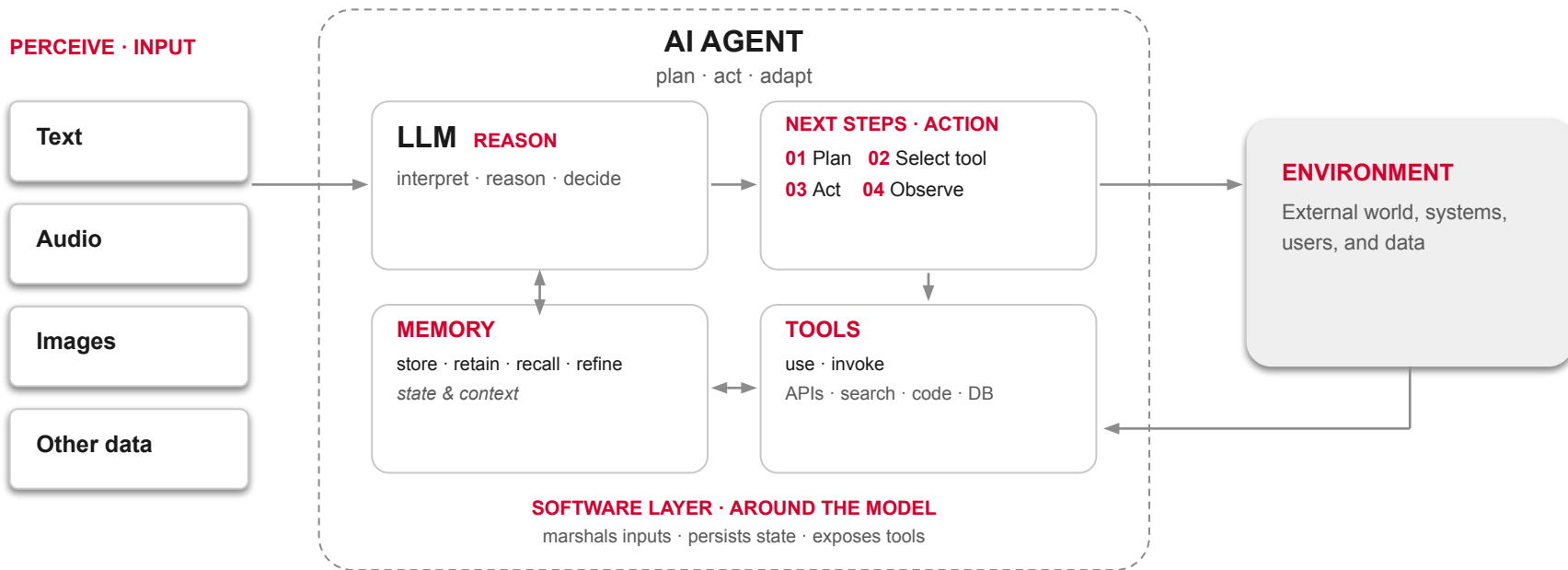
REMEMBERS

persistent information

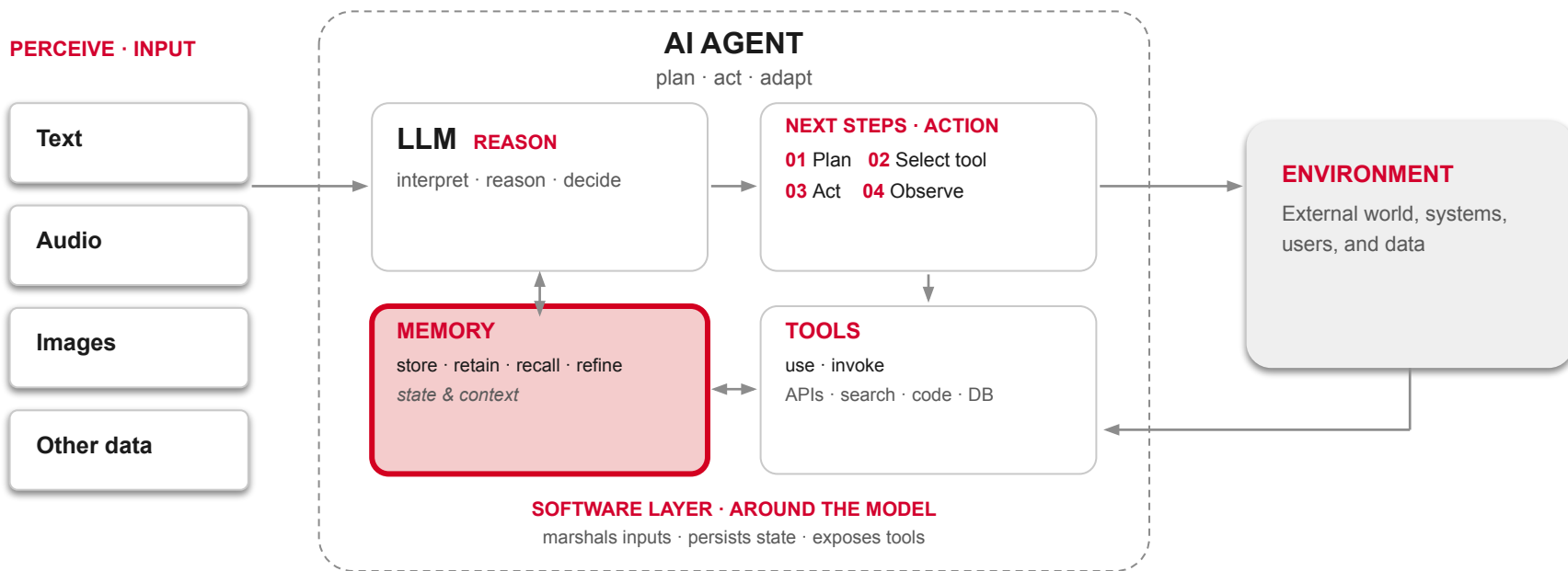
ACTS

through tool use

Anatomy of an AI agent



Anatomy of an AI agent



What Is AI Memory?

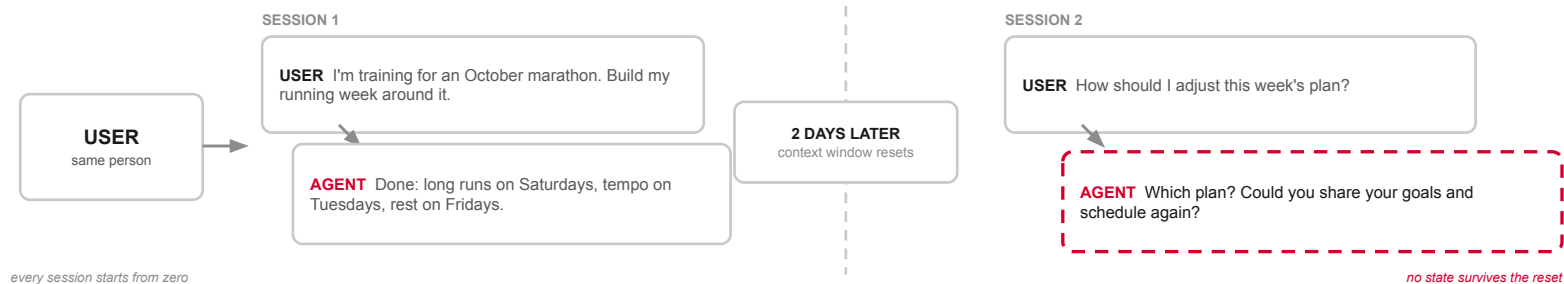
Overview of AI Memory

AI Memory(*in Agentic Context*) == **AI Agent Memory**

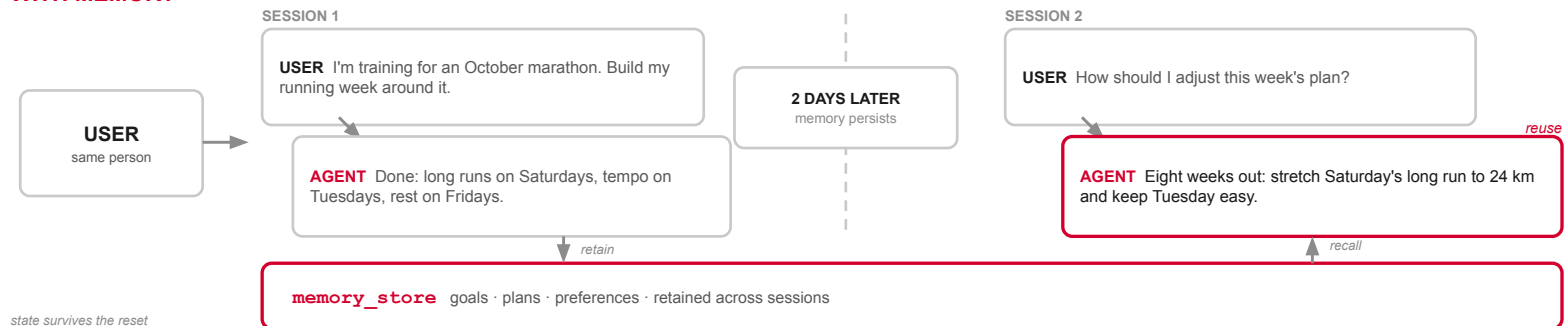


WHY AGENTS NEED MEMORY

WITHOUT MEMORY



WITH MEMORY





What Is Agent Memory?

Overview of AI Memory

Agent memory /ˈeɪ.dʒənt ˈmem.ər.i/ noun

from the Latin **agere** (to act) + **memoria** (mindful, remembering)

- 1 Agent memory is the capability that lets an agent **retain** its experience beyond the context window, **recall** what matters when it matters, **reuse** it to shape behaviour, and **refine** it as the world changes, so future behaviour improves **without resupplying the entire past**.
- 2 *(in this course)* The **system of record for experience**: what survives the reset, governed across its lifecycle.

memory = state + policy

RETAINS

experience beyond the window

RECALLS

what matters, when it matters

REUSES

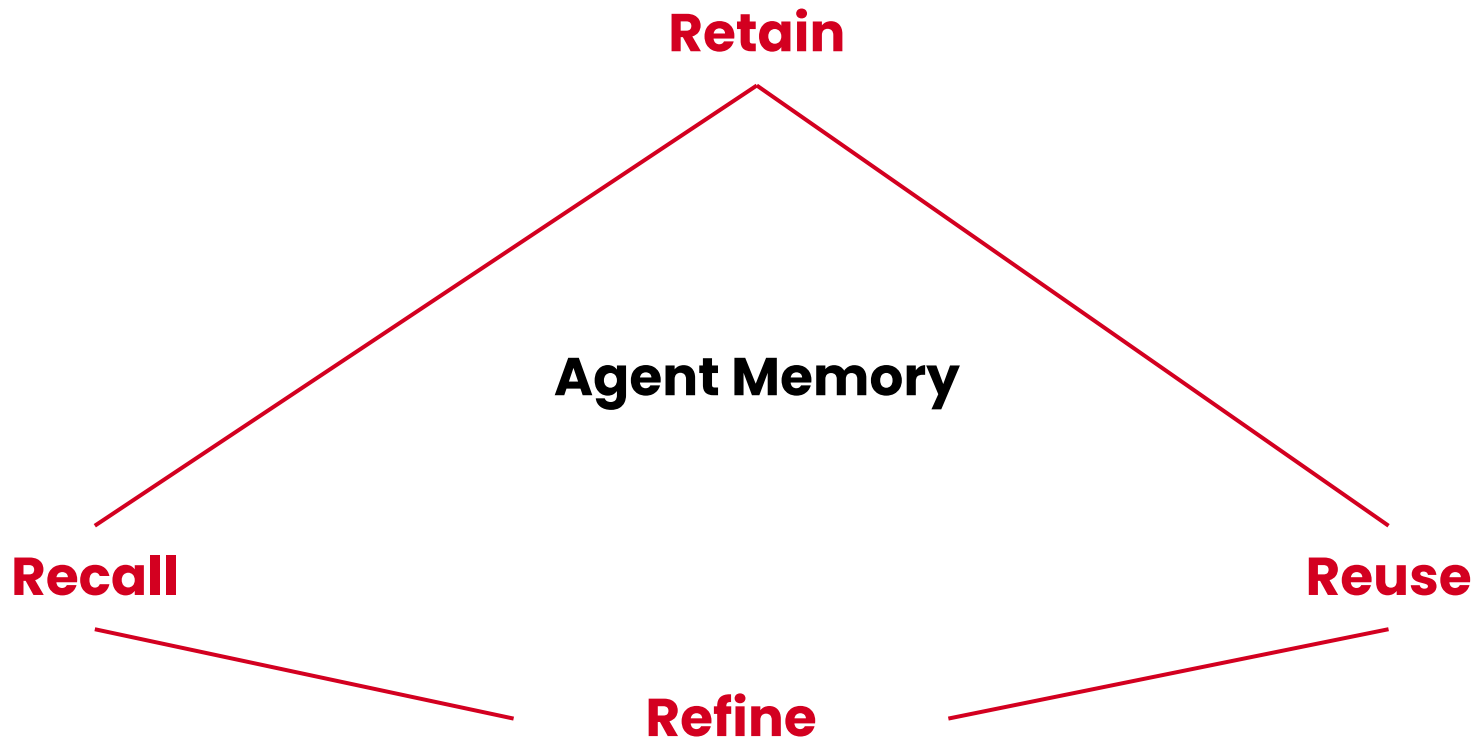
recall becomes behaviour

REFINES

the store improves with hindsight

AI Memory == Agent Memory

Overview of AI Memory



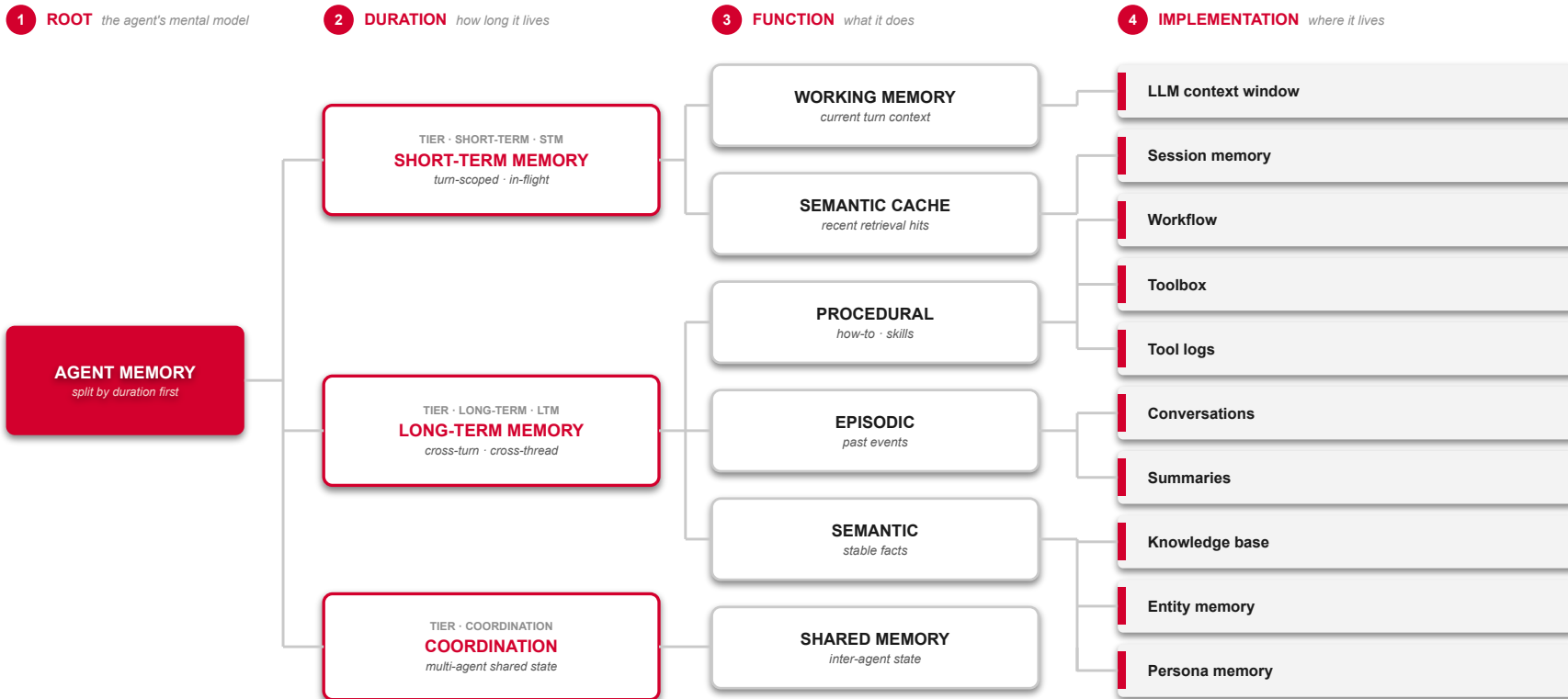


Taxonomy



Forms of Memory in AI Agents (Memory Types)

Overview of AI Memory





The Memory A Model Ships With

Overview of AI Memory

PARAMETRIC MEMORY *model weights*

- knowledge encoded in the parameters, at training time
- fine-tunes update it: slowly, globally, expensively
- static during inference: nothing learned mid-task

CONTEXT MEMORY *the window*

- the current conversation and working state
- token-limited: attention is the budget
- lost when the turn ends

THE GAP bounded window · stateless inference: the model cannot remember you, only what you just showed it.

HOW LONG EACH SURVIVES

PARAMETRIC

WINDOW

EXTERNAL STORE

written at training · lives for years, read-only

written per call · gone in minutes

written by you · lives as long as policy says

IN THE LOOP Weights are slow and frozen; the window is fast and forgotten. Agent memory is everything built to live in between.

Agent Memory, Assembled

Overview of AI Memory



`agent_memory` =

LLM MEMORY

+

EXTERNAL PERSISTENCE

+

MANAGEMENT SYSTEM

LLM MEMORY

given · ships with the model

- parametric knowledge from training
- the per-call working window
- nothing that survives the reset

EXTERNAL PERSISTENCE

built · the store

- short and long-term storage
- cross-session continuity by design
- the types tree lives here: episodic, semantic, procedural, cache, shared

MEMORY MANAGEMENT

run · the lifecycle

- extract · consolidate · retrieve · evolve
- learning from experience; behavioural adaptation
- dynamic: written, refined, and forgotten by policy

RELIABLE consistent context across sessions

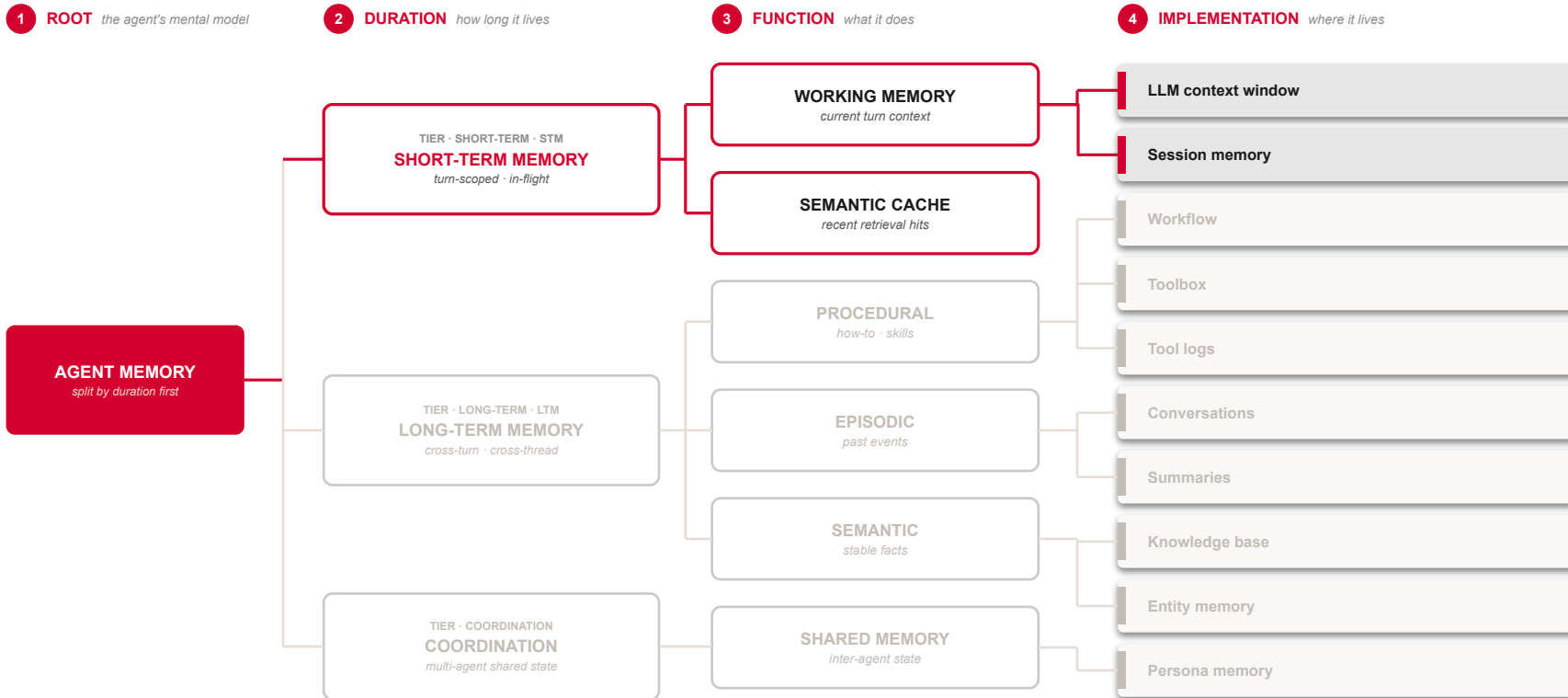
BELIEVABLE trustworthy, grounded interactions

CAPABLE knowledge that accumulates

IN THE LOOP The first term ships with the model. The other two are the job: the store you build, and the lifecycle you run.

Short-Term Memory

Overview of AI Memory





What Is Short-Term Memory?

Overview of AI Memory

short-term memory /ˌʃɔːt.tɜːm ˈmem.ər.i/ noun

from the Latin **memoria**: mindful, remembering · named as a store by **Atkinson & Shiffrin, 1968**

- 1 Short-term memory is the information layer agents use for **immediate task execution**, typically persisting from **seconds to hours**: the tier that holds **work in flight**, spanning the agent's active scratchpad and its recently answered queries.
- 2 *(in this course)* The **turn-scoped tier**: working memory plus the semantic cache, gone or graduated when the session ends.

stm = scratchpad + cache

HOLDS

the work in flight, this turn

CACHES

recent answers, reused by meaning

EXPIRES

seconds to hours, by design

GRADUATES

promotion to LTM is a decision



Short-Term Memory, Defined

Overview of AI Memory

TIER · SHORT-TERM

Short-term memory is the information layer agents use for **immediate task execution**, typically persisting from **seconds to hours**. It holds the work in flight: the active scratchpad and the recently answered.

WORKING MEMORY

the agent's active scratchpad: intermediate results, current observations, in-progress computation

SEMANTIC CACHE

recent query and response pairs, indexed by meaning for instant reuse

both expire by design: promotion to long-term memory is a decision, not a default

IN THE LOOP Seconds to hours: fast, cheap, disposable. What deserves to outlive the session is the write path's call, not STM's.



Working Memory

Overview of AI Memory



STM · FUNCTION 01

Working memory is the agent's **temporary workspace** for manipulating information during active tasks: a scratchpad holding **intermediate results, current observations and in-progress computation.**

IMPLEMENTATION A · CONTEXT WINDOW PARTITION

a strategically managed segment of the active prompt: what stays in, what gets compressed, what gets flushed

IMPLEMENTATION B · SESSION STORAGE

keyed temporary state outside the window, injected selectively; persists through the session, cleared when it ends



Working Memory In The Window

A worked token budget · 8,000-token context

CONTEXT WINDOW · 8,000 TOKENS

SYSTEM PROMPT

role: technical support agent · policies · tone

500

CONVERSATION HISTORY · COMPRESSED

older turns summarised: Q2 results discussion, condensed

2,500

WORKING MEMORY SEGMENT

current task: analyse Q3 earnings · AAPL · sector: technology

2,000

CURRENT QUERY

projected impact on next quarter's earnings?

200

AVAILABLE SPACE

headroom for retrieval and generation

~2,800

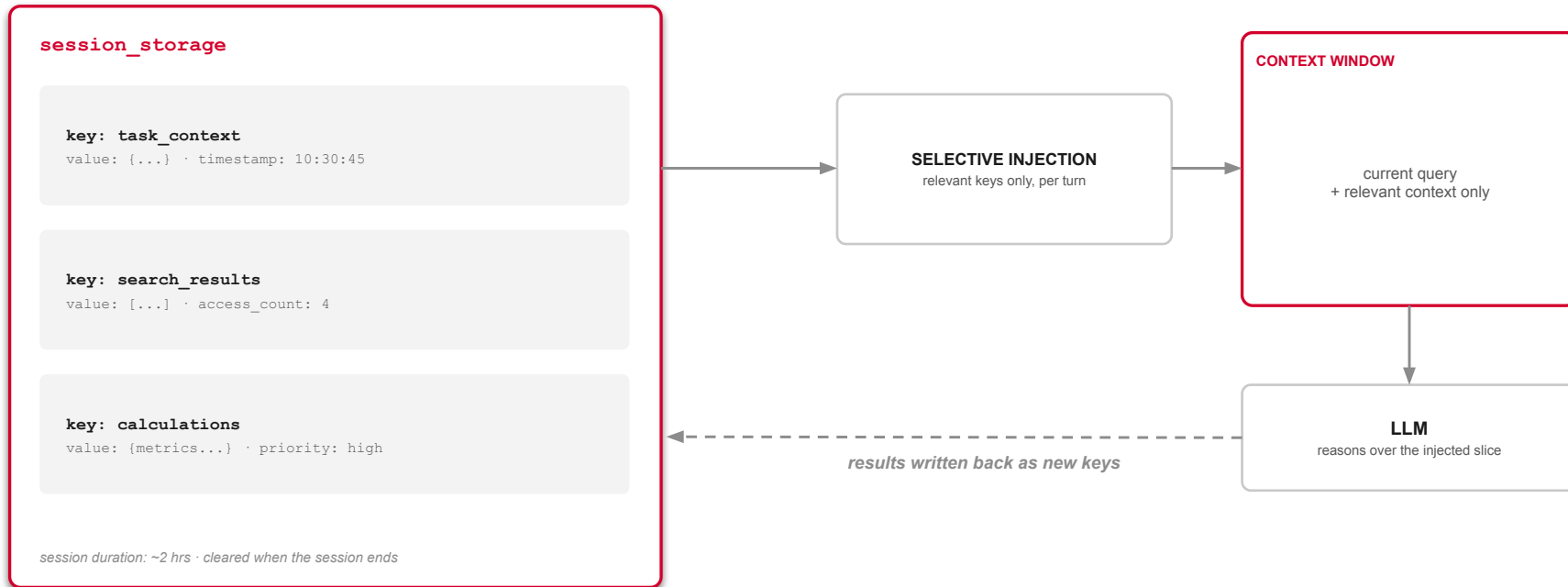
AS THE WINDOW FILLS

- older working memory is compressed into summaries
- low-priority information is flushed from the context



Working Memory As Session Storage

Overview of AI Memory

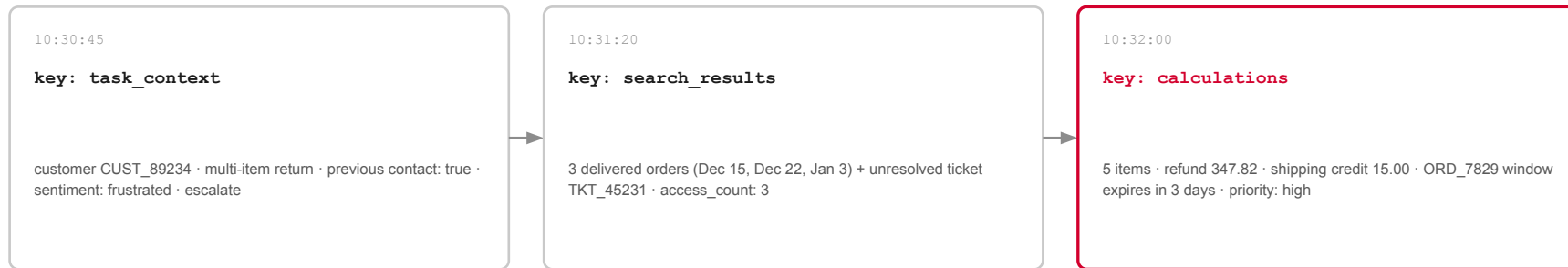


Working Memory In Action

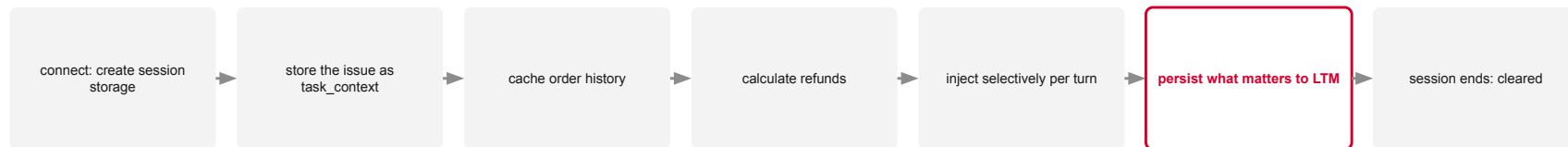
Case study · e-commerce support agent, multi-order return



Sarah is returning items from three orders, has explained it twice already, and got disconnected yesterday. Frustrated, and the return windows are ticking.



each turn reads and writes these keys: access counts and priorities steer what gets injected



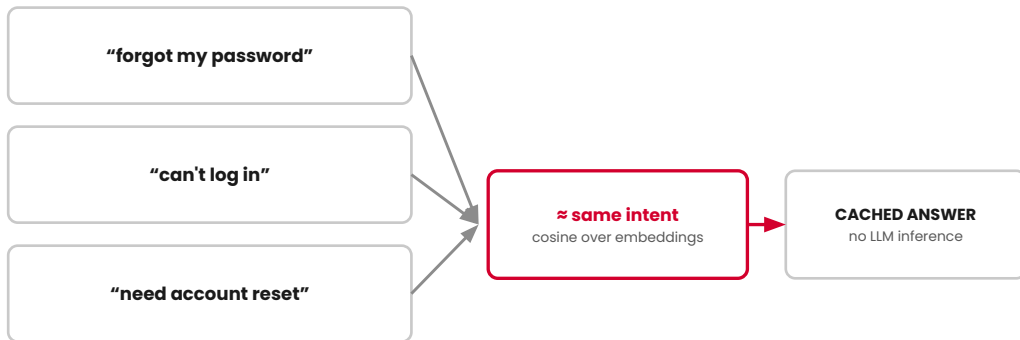
step six is the handover: the resolution and preferences graduate to long-term memory before the scratchpad is wiped

IN THE LOOP One session, three keys, seven steps. Working memory carries the case; long-term memory keeps only what earned it.

Semantic Cache



Semantic cache stores recent **query and response pairs indexed by meaning**, not exact text: vector embeddings identify when a new query carries the **same underlying intent** as one already answered, and return the cached response without a fresh inference.



guards before a hit: freshness · tenant identity · version fingerprint (module 4 goes deeper)

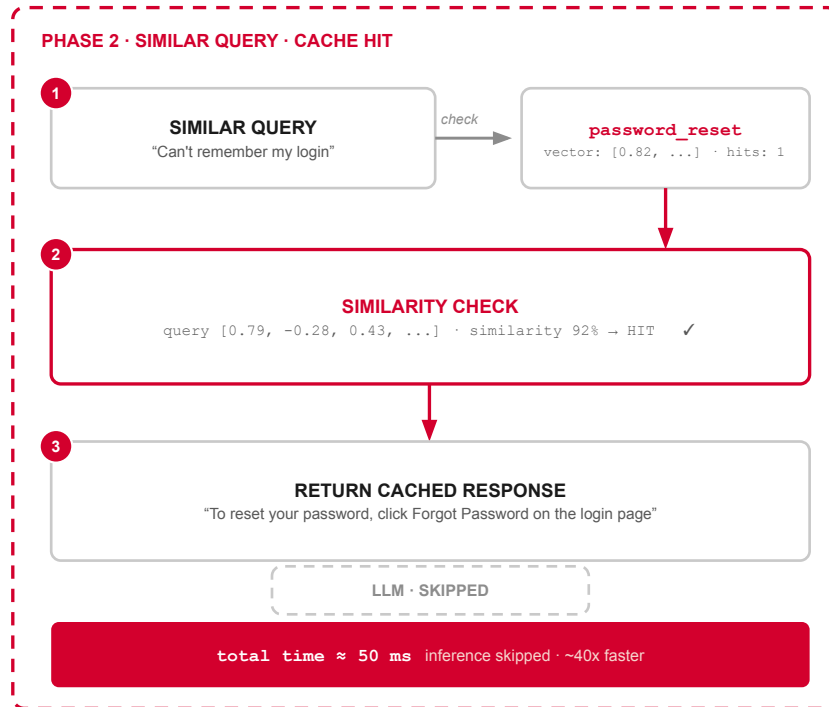
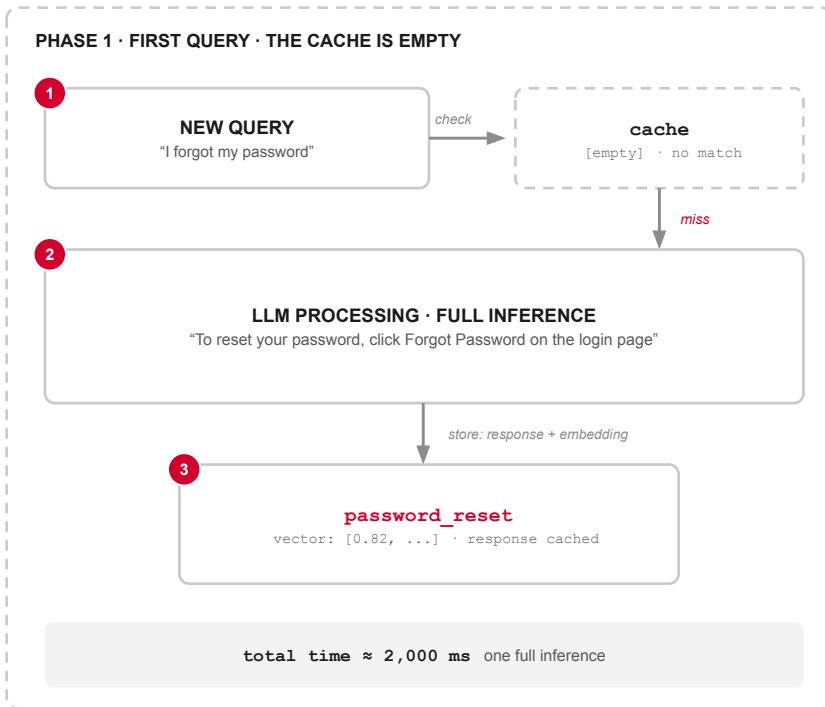
WHY IT IS SHORT-TERM

hits decay as products, policies and models change; the cache is rebuilt from live traffic, not archived



The Semantic Cache Lifecycle

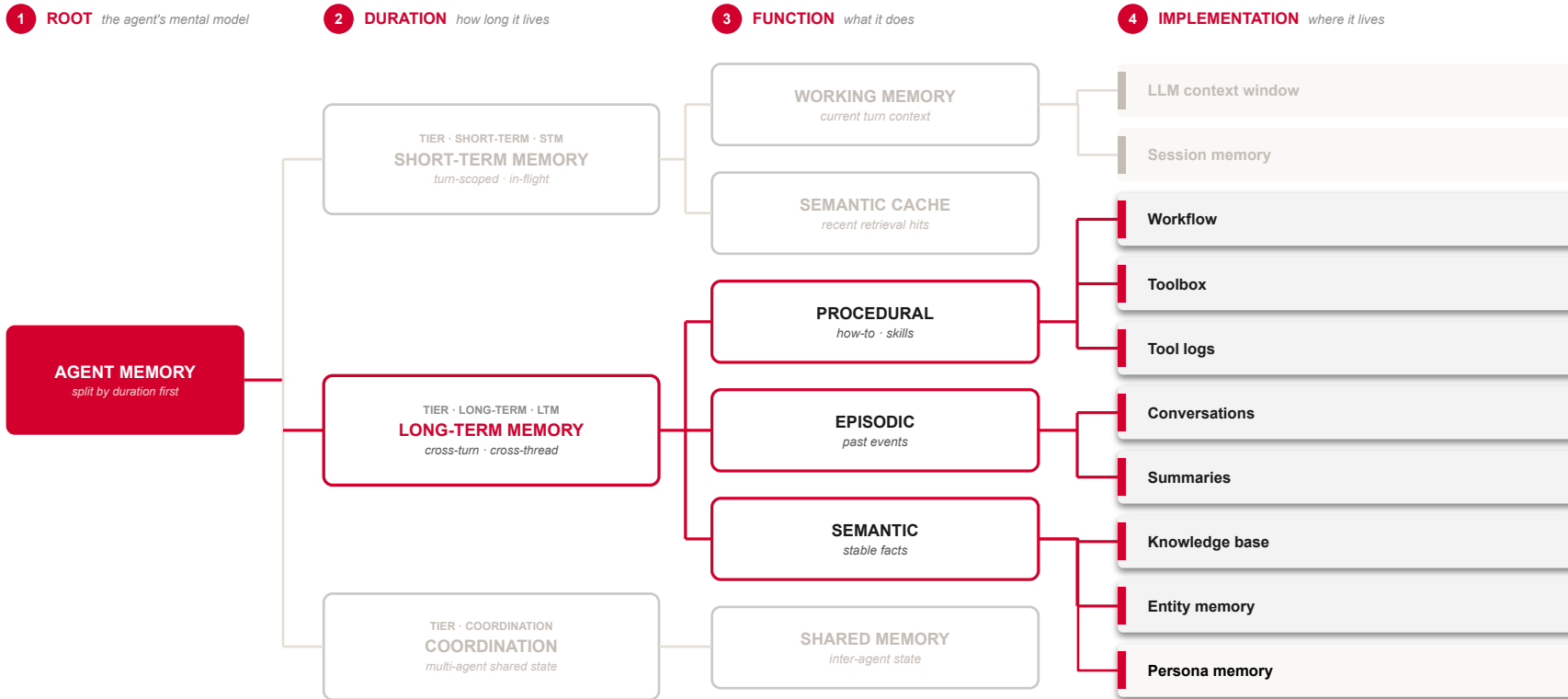
Overview of AI Memory



IN THE LOOP Two seconds the first time, fifty milliseconds ever after: the cache pays for itself on the second question.

Long-Term Memory

Overview of AI Memory





What Is Long-Term Memory?

Overview of AI Memory

long-term memory /ˌlɒŋ.tɜːm ˈmem.ər.i/ noun

from the Latin **memoria**: mindful, remembering · named as a store by **Atkinson & Shiffrin, 1968**

- 1 Long-term memory is the **persistent knowledge foundation** that enables agents to **maintain continuity, accumulate learning, and evolve capabilities** across **sessions and extended time periods**.
- 2 *(in this course)* **Skills, stories and facts**: the cross-session tier, procedural, episodic and semantic over one substrate.

Itm = skills + stories + facts

PERSISTS

cross-turn, cross-thread, by design

ACCUMULATES

knowledge compounds with use

EVOLVES

capability improves from experience

FORGETS

by policy, never by accident



Long-Term Memory, Defined

Overview of AI Memory



TIER · LONG-TERM

Long-term memory is the **persistent knowledge foundation** that lets agents maintain **continuity, accumulate learning and evolve capability** across sessions and extended time. It is where the four Rs stop being aspiration and become storage.

PROCEDURAL

how to do things: workflows, tools, learned skills

EPISODIC

what happened: conversations and their summaries

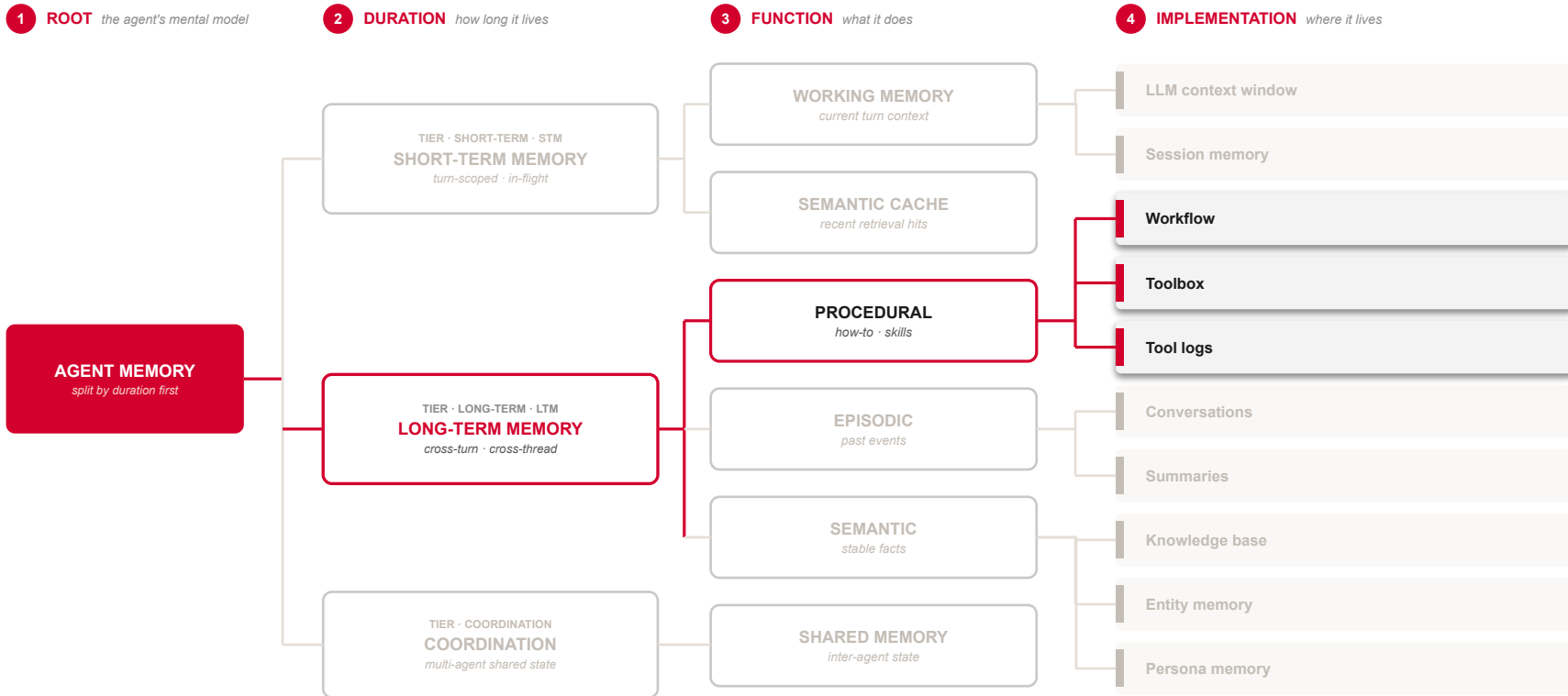
SEMANTIC

what is known: facts, entities, the knowledge base



Procedural Memory

Overview of AI Memory



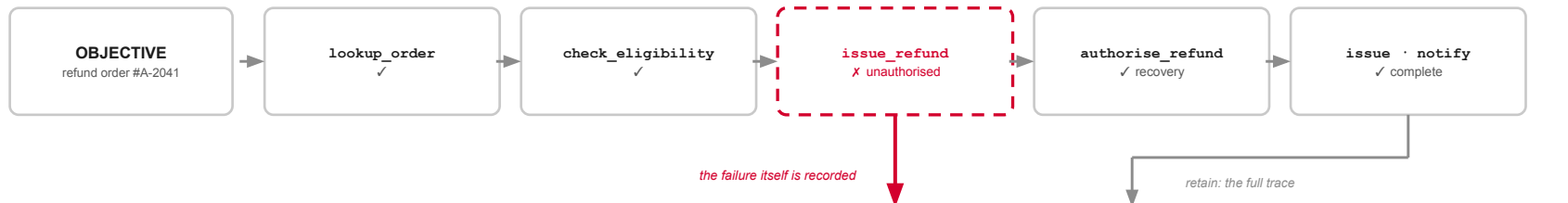
IN THE LOOP How-to memory: workflows, the toolbox, and the tool logs that feed both.



Workflow Memory: Failure Is Experience Too

Type of Procedural Memory in AI Agents

RUN 1 · FIRST TIME



RUN 2 · SIMILAR OBJECTIVE · GUIDED

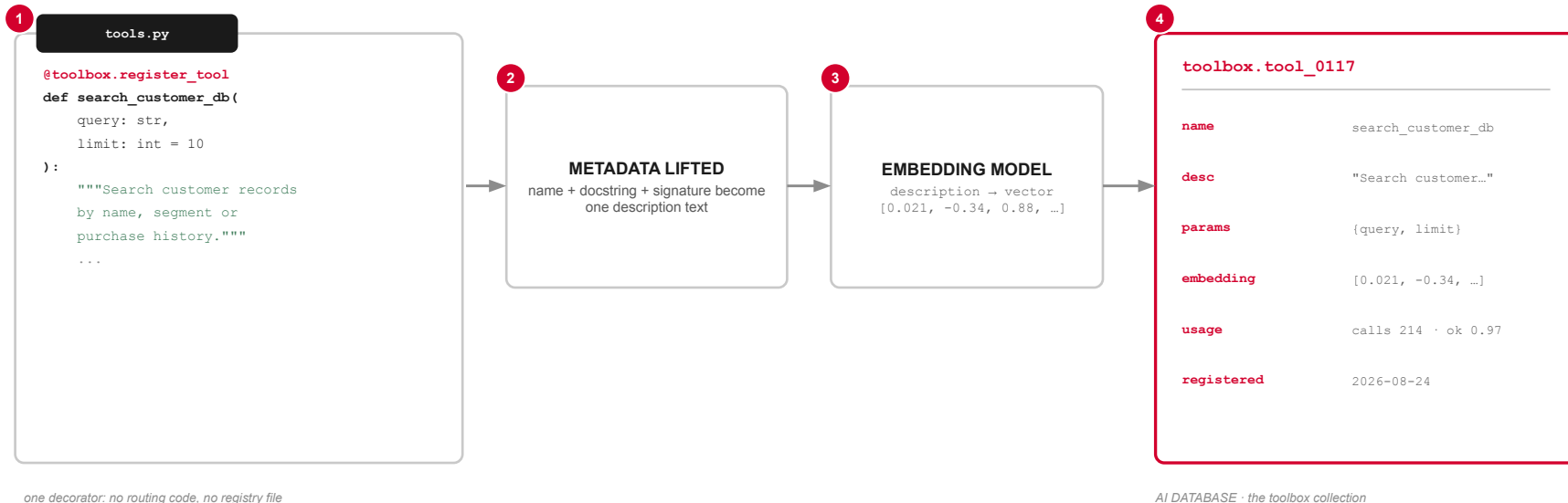


IN THE LOOP The first run pays the tuition; every similar run inherits the lesson. Successes are patterns, failures are guards.



Toolbox Memory: Registering A Tool

Type of Procedural Memory in AI Agents



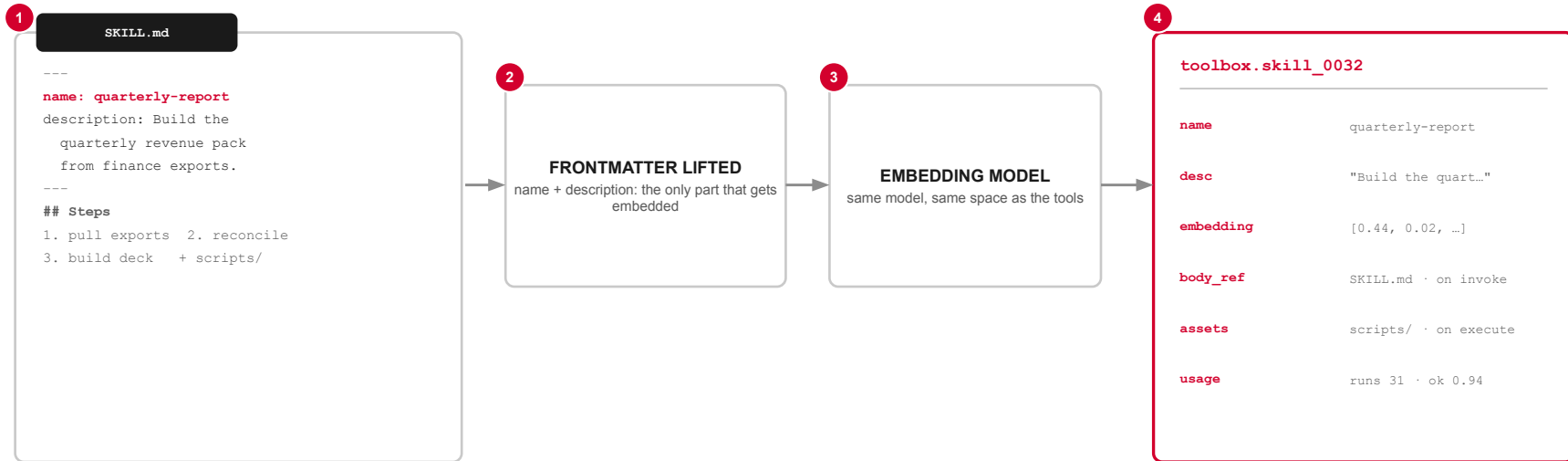
FROM HERE, DISCOVERY IS RETRIEVAL a task arrives as a vector, semantic match finds this record above 0.8, and only then does the schema enter the window.

IN THE LOOP Write the function, add one decorator, and the capability is findable by meaning: registration is the whole integration.



The Same Registry Extends To Skills

Type of Procedural Memory in AI Agents



a procedure as a file: instructions, steps, assets

AI DATABASE · the same capability store

ONE CAPABILITY STORE, SEARCHED BY MEANING

index in the window · body on invoke · assets on execute: progressive disclosure

tool · search_customer_db

tool · run_sql

skill · quarterly-report

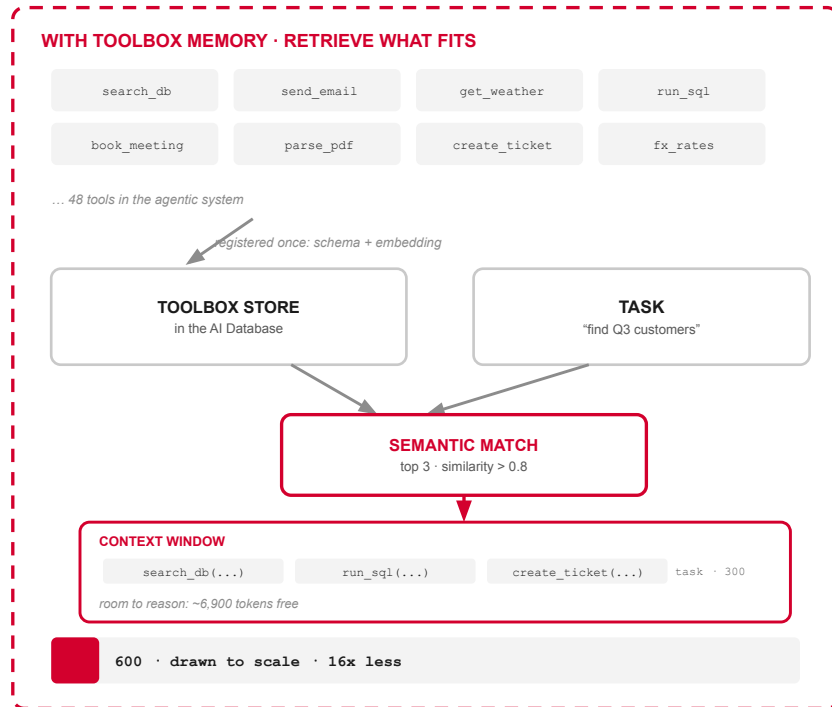
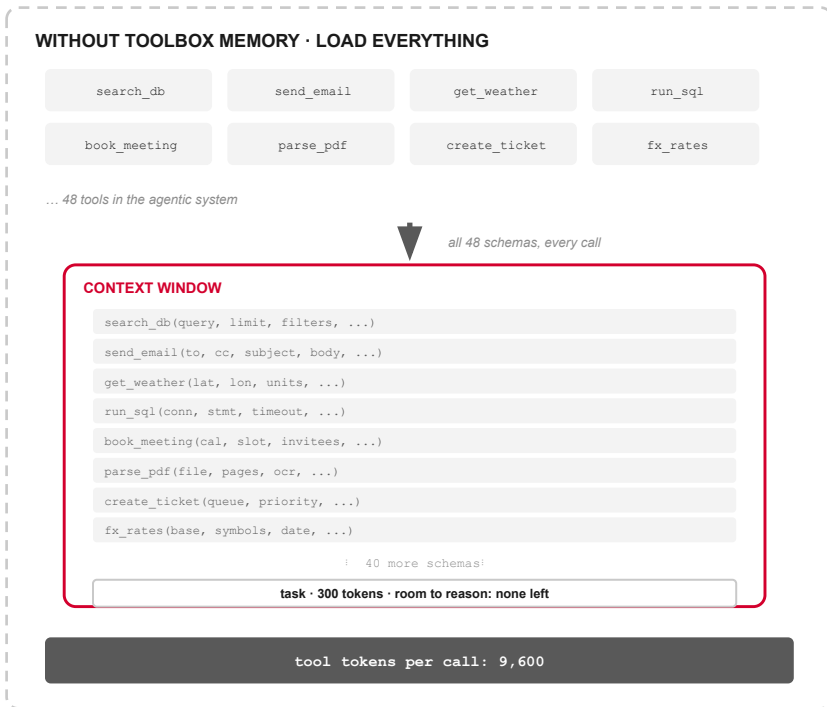
skill · onboarding-pack

IN THE LOOP Tools and skills are the same shape to the registry: a name, a description, a vector. Only what happens after retrieval differs.



Toolbox Memory: Load What Fits, Not What Exists

Type of Procedural Memory in AI Agents

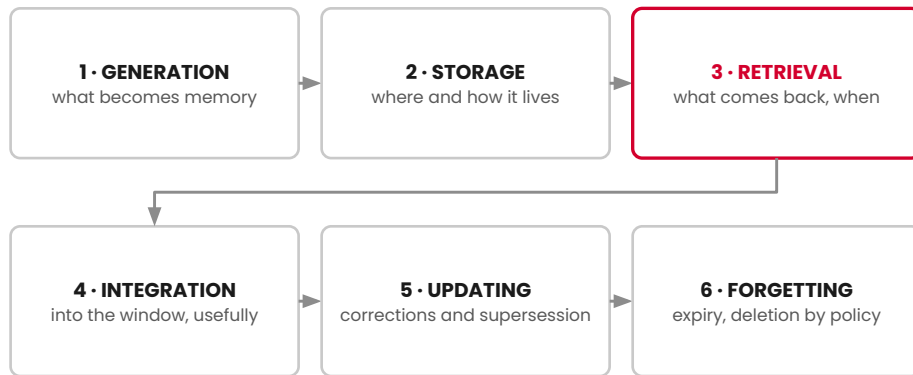


IN THE LOOP Loading every tool taxes every token after it. Retrieval loads the three the task can use, and the window breathes.



Memory Management Is Systematic

Overview of AI Memory



WITHOUT SYSTEMATIC COORDINATION

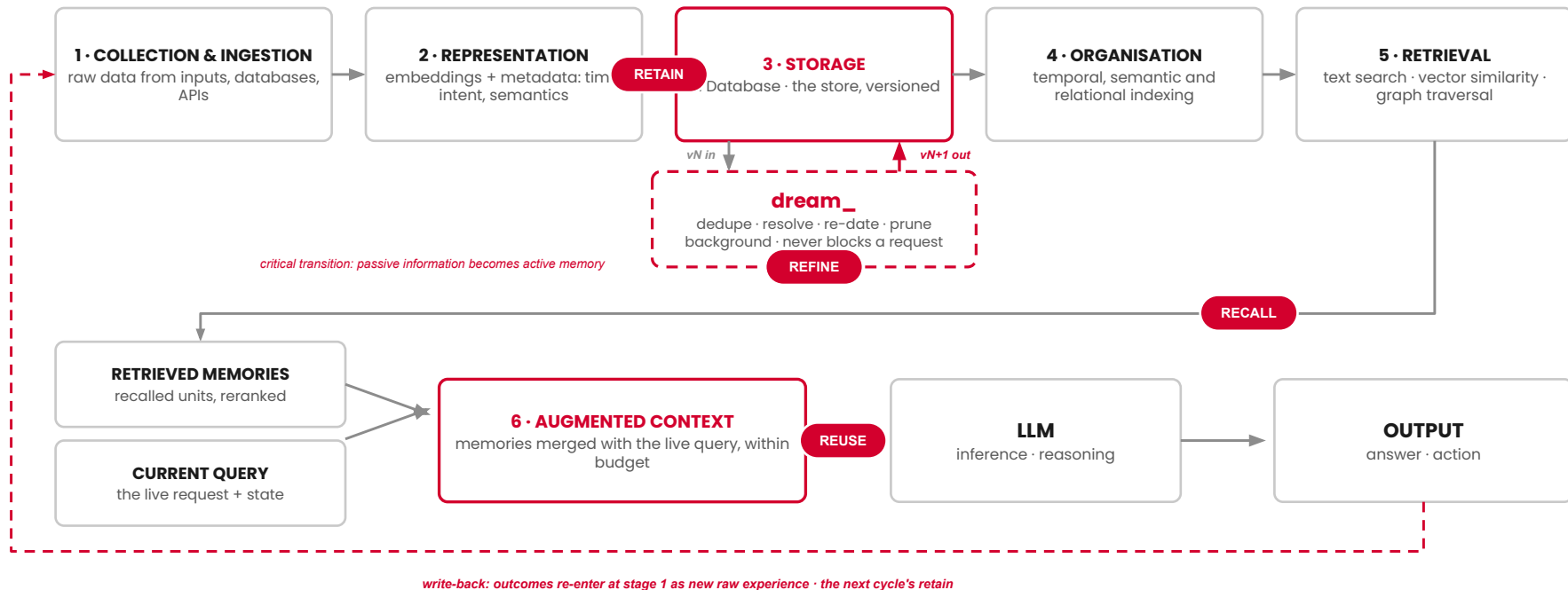
- retrieval surfaces outdated information that consolidation should have retired
- storage duplicates what semantic memory already holds
- forgetting removes context current operations still need

IN THE LOOP Systematic because memory IS the system that turns a stateless model into an agent; the operations only work as one loop.



The Memory Engineering Lifecycle

From raw data to active memory, and back



The Importance and Benefit of Memory in AI Agents

Overview of AI Memory



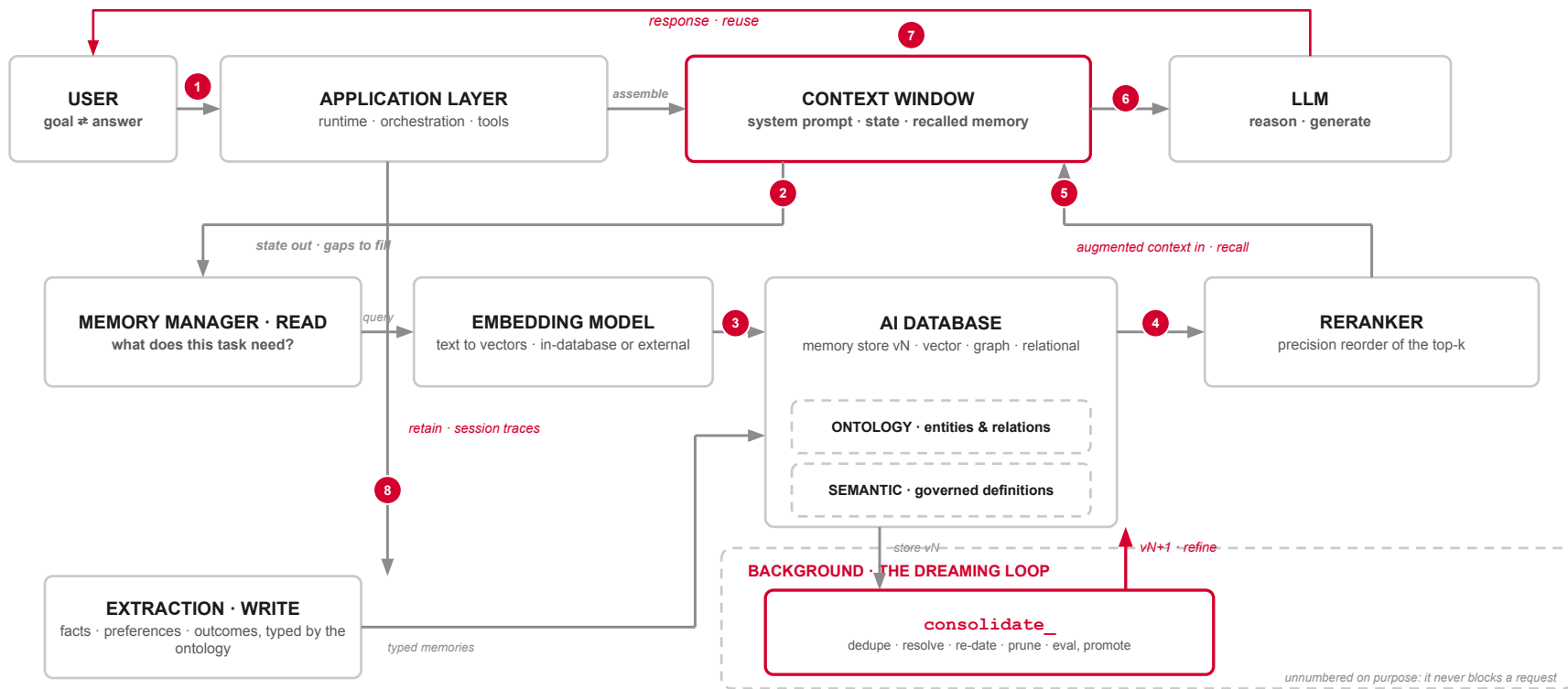


Agent Memory Architecture



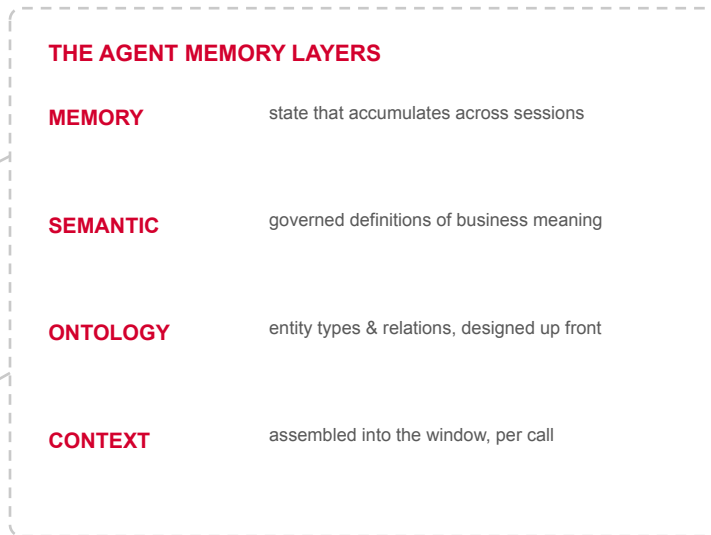
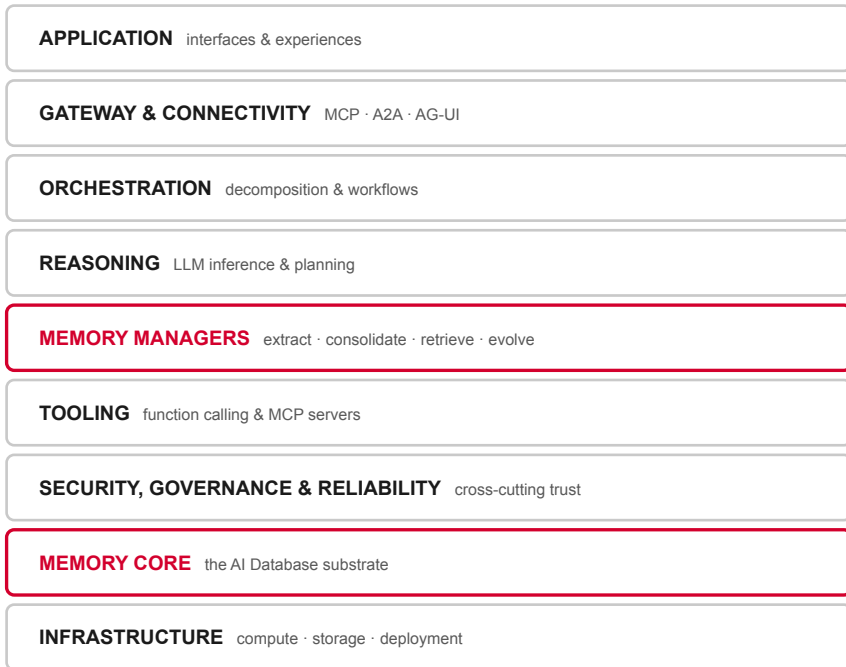
Agent Memory, A Reference Architecture

Overview of AI Memory



The Agent Stack

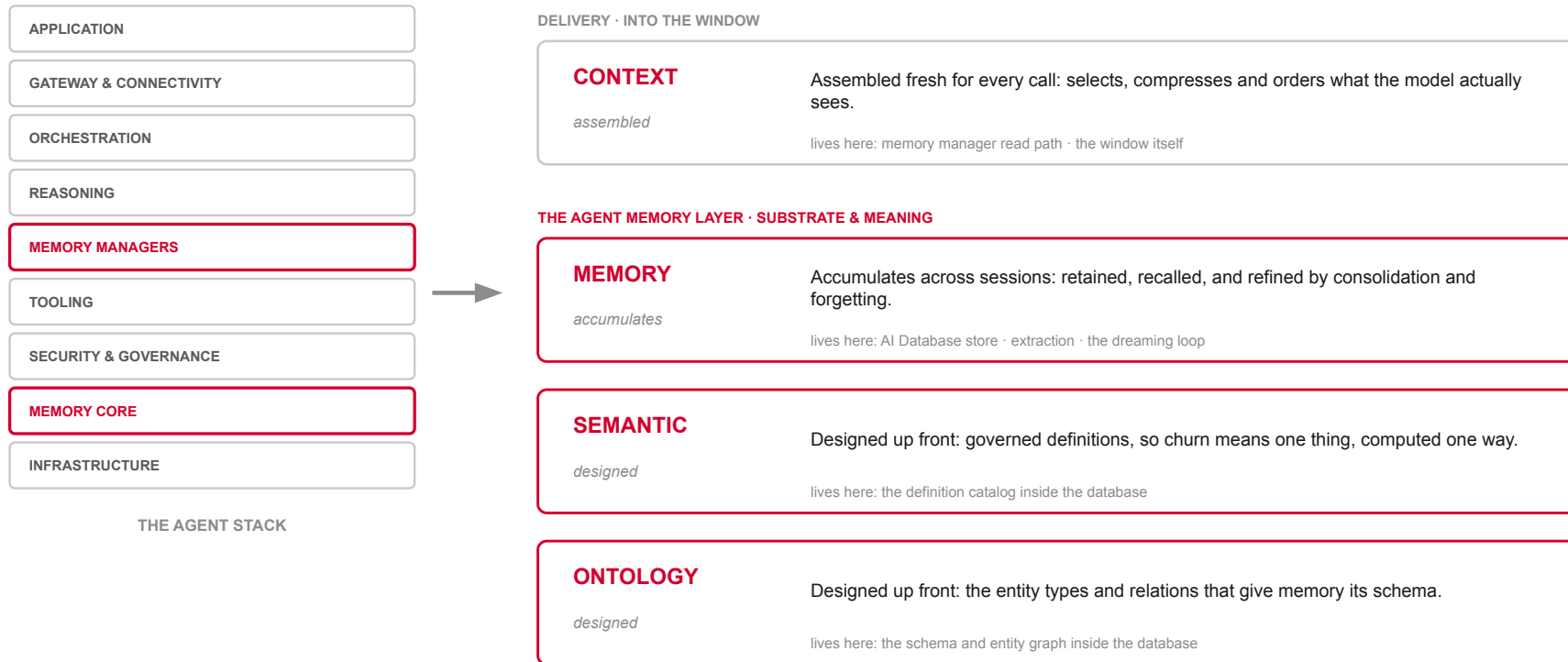
Overview of AI Memory





Inside The Agent Memory Layer

Overview of AI Memory

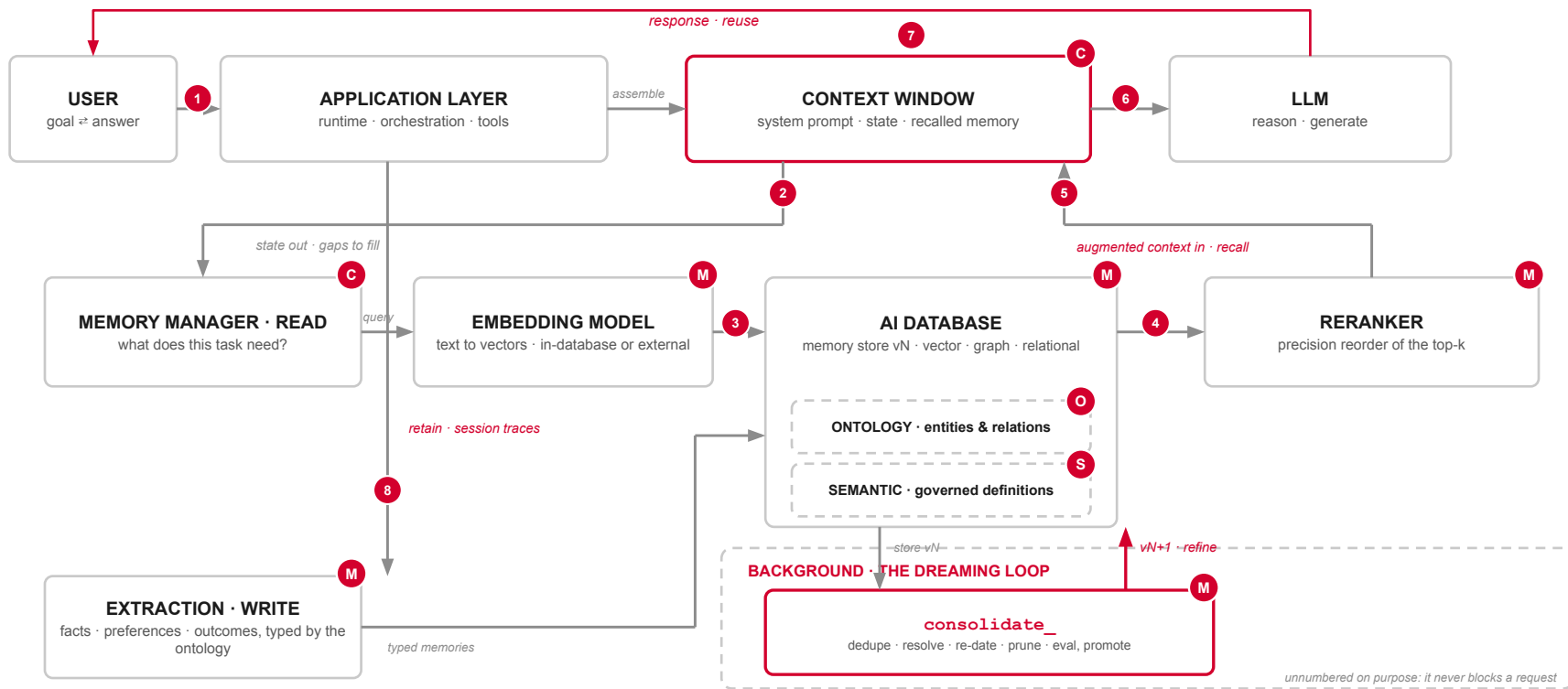


IN THE LOOP Ontology and semantic are designed, memory accumulates, context is assembled: meaning, then state, then attention.



Where The Layers Live

Mapping the Agent Stack onto the architecture





What Is The Memory Layer?

Overview of AI Memory

memory layer /'mem.ər.i 'leɪ.ə/ noun

from the Latin **memoria**: mindful, remembering

- 1 The memory layer is the **persistence substrate** of an agent: the state that **accumulates across sessions**, retained beyond the context window, recalled when relevant, and **refined** by consolidation and forgetting.
- 2 *(in this course)* The **system of record for experience**: what survives the reset.

memory = store + lifecycle

RETAINS

experience beyond the session

RECALLS

retrieval lives here

REFINES

consolidation: the dreaming loop

FORGETS

by policy, not by accident



What Is The Semantic Layer?

Overview of AI Memory

semantic layer /sɪˈmæn.tɪk ˈleɪ.ə/ noun

from the Greek **sēmantikos**: significant, giving signs

- 1 The semantic layer binds the concepts an agent reasons with to **governed definitions** of the data underneath, so churn, revenue and an active customer mean **one thing, computed one way**, everywhere the agent touches them.
- 2 *(in this course)* The **dictionary of business meaning** that keeps remembered facts trustworthy.

concepts = governed definitions

DEFINES

concepts against real data

GOVERNS

one calculation, everywhere

GROUNDS

text-to-SQL & tool selection

≠ SEMANTIC MEMORY

the memory type shares a word, nothing else



What Is The Ontology Layer?

Overview of AI Memory

ontology layer /ɒnˈtɒl.ə.dʒi ˈleɪ.ə/ noun

from the Greek *ōn*, *ontos* + *logia*: the study of what exists

- 1 The ontology layer is the agent's **model of what exists**: the **entity types, relationships and constraints** that give memory its schema, so R. Alake and Richmond resolve to **one node, not two strings**.
- 2 *(in this course)* Designed **before anything is remembered**: the shape a valid memory must fit.

ontology = entities + relations

TYPES

entities, not strings

RELATES

the graph between facts

RESOLVES

R. Alake and Richmond, one node

CONSTRAINS

what a valid memory is



What Is The Context Layer?

Overview of AI Memory

context layer /ˈkɒn.tekst ˈleɪ.ə/ noun

from the Latin **contexere**: to weave together

- 1 The context layer decides **what the model actually sees: assembled fresh** for every inference call, it selects, compresses and orders material from memory against a **finite token budget**.
- 2 *(in this course)* Working memory **on a deadline**: the window is spent every turn.

context = assembled, per call

SELECTS

what earns its tokens

COMPRESSES

summaries over transcripts

ORDERS

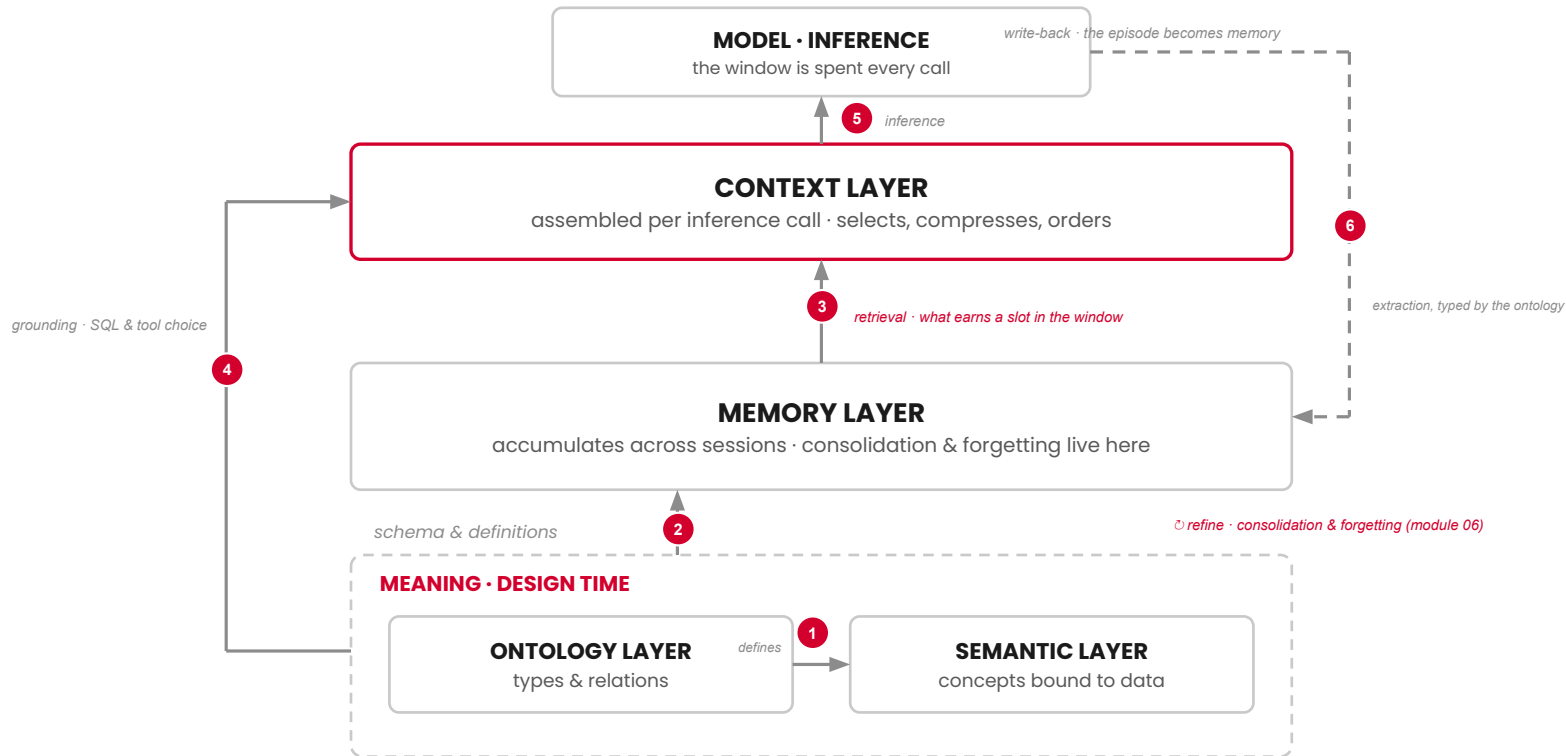
position shapes attention

EXPIRES

rebuilt every call

How The Layers Relate

Overview of AI Memory





From Layers to Discipline



The Engineering Disciplines In AI Agent Memory

Overview of AI Memory



MEMORY ENGINEERING

manages information beyond the window · persistent, adaptive

CONTEXT ENGINEERING

manages information within the window · assembled per call

PROMPT ENGINEERING

crafts the instructions and queries
single turn · the words themselves



What Is Prompt Engineering?

Overview of AI Memory

prompt engineering /ˌprɒpt en.dʒɪˈniə.rɪŋ/ noun

from the Latin *promptus*: brought forth, ready at hand

- 1 Prompt engineering is the practice of **designing and refining** the instructions, system messages and inputs passed to a language model, so that its **probabilistic generation** correlates with the **intent stated in the prompt**.
- 2 *(in this course)* The **words themselves**: single turn, foundational, and brittle on its own.

prompt = instructions + intent

INSTRUCTS

persona, policies, output contracts

DEMONSTRATES

few-shot: show, do not tell

STRUCTURES

reasoning steps and answer shape

RANKS

instruction hierarchy resolves conflicts



Prompt Engineering: The Techniques

Overview of AI Memory

INSTRUCTIONS & SYSTEM MESSAGES

persona, policies, tone: the contract the model works to

FEW-SHOT EXAMPLES

show, do not tell: worked examples beat description

CHAIN OF THOUGHT

reason before answering; think steps, then speak

INSTRUCTION HIERARCHY

system > developer > user > tool;
conflicts resolve upward

FAILURE-MODE HANDLING

say what to do when things break, before they do

STRUCTURED OUTPUT

schemas, formats and contracts the caller can parse

DECODING CONTROLS

temperature and sampling discipline per task

REACT PATTERNS

interleave reasoning with action and observation



What Is Context Engineering?

Overview of AI Memory

context engineering /'kɒn.tekst en.dʒɪ'niə.rɪŋ/ noun

from the Latin **contexere**: to weave together

- 1 Context engineering is the systematic practice of **dynamically constructing** the information fed into the model's context window, asking **what configuration of context** is most likely to produce the desired behaviour within a **finite token budget**.
- 2 *(in this course)* The **window, filled deliberately**: assembled fresh, every token earning its place.

context = configuration > wording

ASSEMBLES

dynamic construction, per call

RETRIEVES

just-in-time relevance

COMPRESSES

reduction without losing the thread

OFFLOADS

externalise, keep the handle



Context Engineering: The Techniques

Overview of AI Memory

CONTEXT WINDOW UTILITY

optimise attention: position matters, every token earns its place

CONTEXT ORGANISATION

structure for comprehension: sections, tags, ordering

ISOLATION & COMPOSITION

scoped buffers per agent and tool; clean sub-agent handoffs

CONTEXT RETRIEVAL

just-in-time injection of relevance from memory and APIs

CONTEXT REDUCTION

compress without losing coherence; reversible compaction

CONTEXT OFFLOADING

externalise state, keep a handle, rehydrate on demand

PROGRESSIVE DISCLOSURE

names and summaries first, full detail on demand: how skills stay cheap

CACHE-AWARE ASSEMBLY

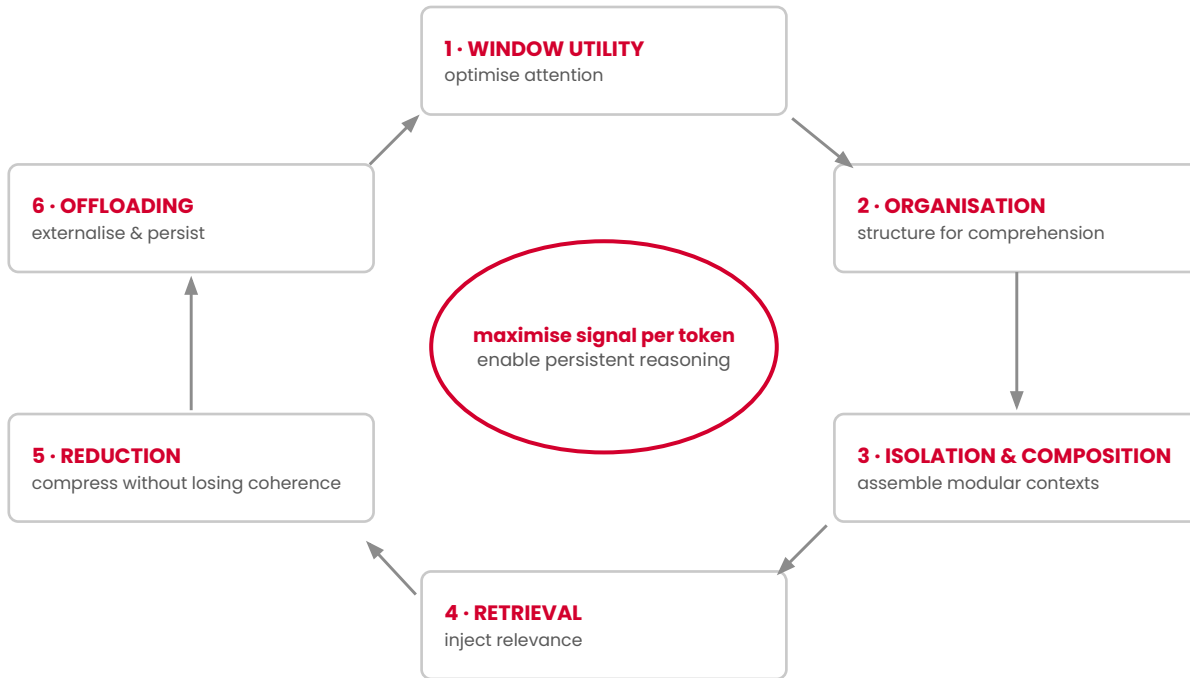
stable prefixes and append-only logs; never break the cache



The Context Engineering Flywheel

Overview of AI Memory

every token must earn its place in the window



measured as token ROI: signal per token spent

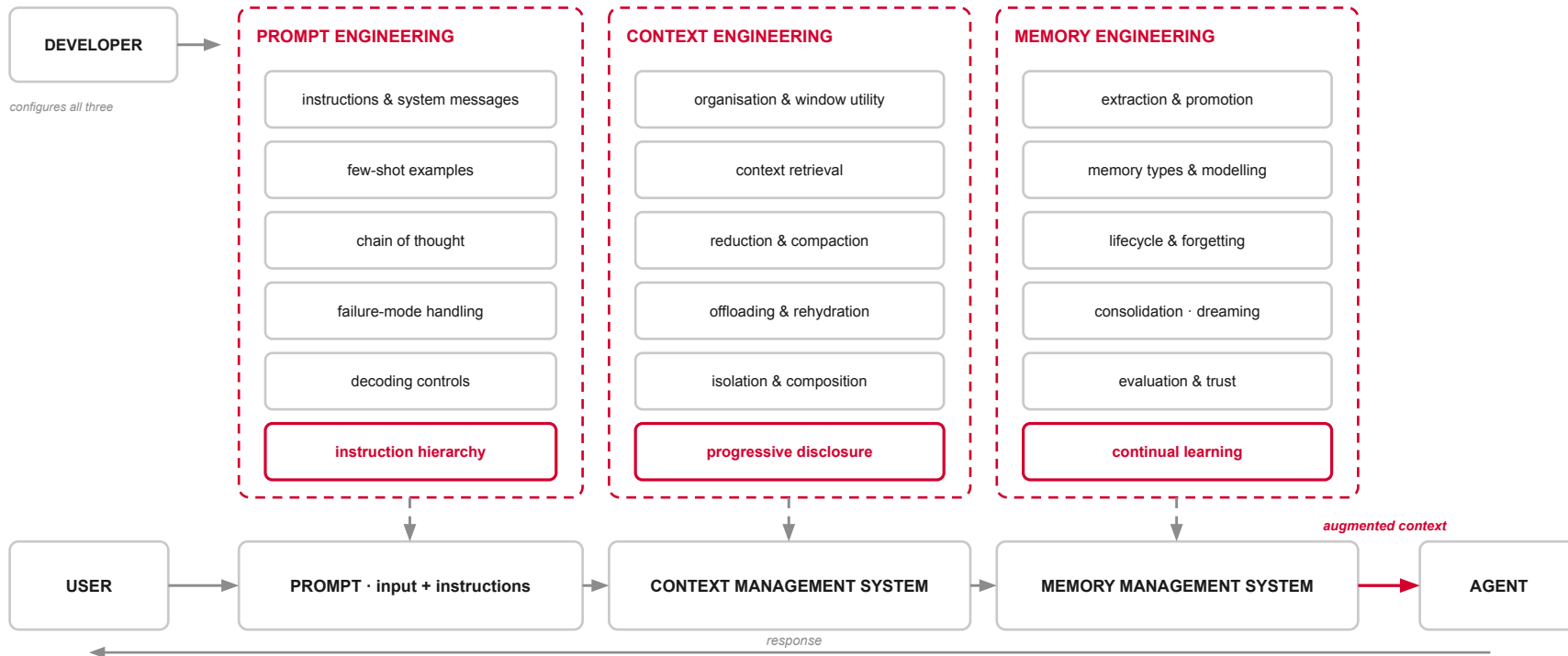
station 6 is the bridge: offloading hands over to memory engineering

IN THE LOOP The wheel spins toward one goal: maximise signal per token, and let reasoning persist beyond the turn.



Three Disciplines, One Pipeline

Overview of AI Memory





Memory Engineering As A Field

Overview of AI Memory

The discipline of designing the systems that let agents retain, recall, reuse and refine experience: the schemas, lifecycles, retrieval paths and consolidation loops that turn transient context into persistent, adaptive knowledge.

SITS BETWEEN

application engineering ⇄ data engineering ⇄ agent engineering ⇄ search optimization

THE AI MEMORY ENGINEER · ROLES & RESPONSIBILITIES

OWN THE SCHEMA

memory unit modelling, ontologies, entity resolution

OWN THE LIFECYCLE

generation to forgetting; forget, do not delete

OWN RECALL

retrieval, reranking, context assembly budgets

OWN THE EVOLUTION

consolidation pipelines and versioned promotion

OWN THE EVIDENCE

memory evals: freshness, continuity, relevance

OWN THE TRUST

provenance, PII, and the poisoning gate



What Is Memory Engineering?

Overview of AI Memory

memory engineering /'mem.ər.i en.dʒi'nɪə.rɪŋ/ noun

from the Latin **memoria**: mindful, remembering

- 1 Memory engineering is the discipline of designing the **cognitive and technical scaffolding** that transforms **transient context into persistent, adaptive knowledge**, governing how information flows between stores, the window, and back, and forming the **foundation for continual learning**.
- 2 *(in this course)* **State under policy**: the eleven-stage lifecycle, owned end to end.

memory = context, made durable

RETAINS

the write path, gated on entry

REFINES

consolidation and dreaming

GOVERNS

provenance, access, forgetting

LEARNS

token space and weight space



Memory Engineering: The Techniques

Overview of AI Memory

MEMORY UNIT MODELLING

typed records shaped by the ontology

EXTRACTION & PROMOTION

decide what deserves to persist

MEMORY TYPE IMPLEMENTATION

persona · toolbox · workflow · knowledge

INDEXING & RETRIEVAL

vector, graph and rerank paths

LIFECYCLE MANAGEMENT

TTL, decay, forget do not delete

CONSOLIDATION & DREAMING

background refine, versioned stores

SEMANTIC CACHE

answered once, reused cheaply

TEMPORAL MANAGEMENT

re-dating, validity windows, graphs

MEMORY EVALUATION

LoCoMo, LongMemEval, promotion gates

MEMORY SECURITY

provenance and the poisoning gate

CONTINUAL LEARNING · TOKEN SPACE

traces promoted into skills

CONTINUAL LEARNING · WEIGHT SPACE

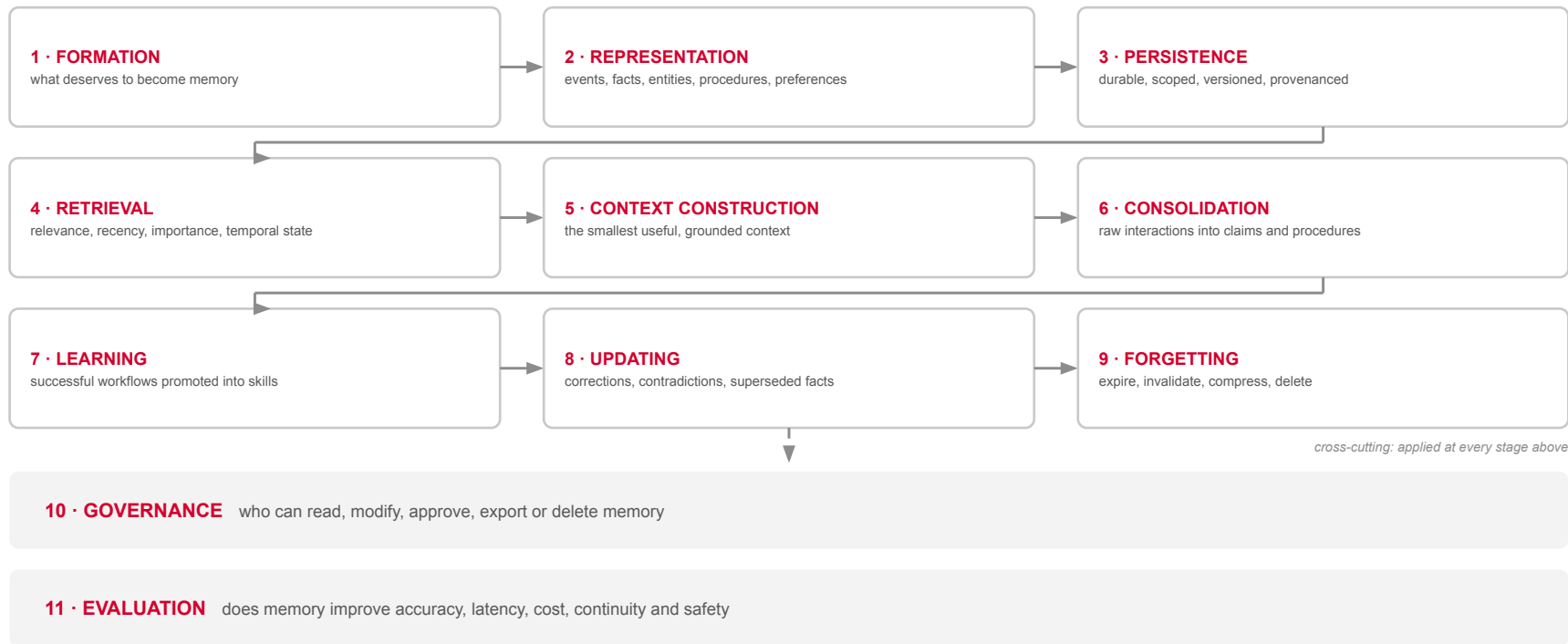
traces distilled into fine-tunes

IN THE LOOP The first ten keep memory healthy; the last two turn it into learning. That is where module 06 hands over.



The Memory Lifecycle, End To End

Overview of AI Memory



IN THE LOOP Eleven stages, one loop. Stages ten and eleven never leave the room: everything above answers to them.



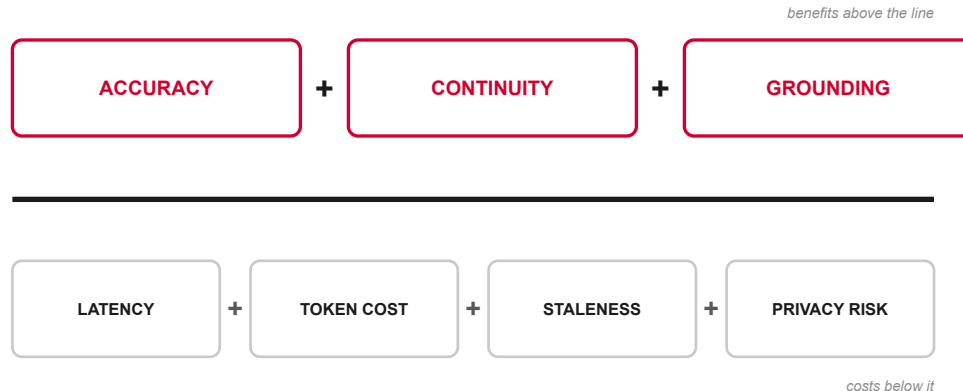
What A Memory Engineer Optimises

Overview of AI Memory

Agent memory is how we make intelligent systems retain, reuse, recall and refine information so that future behaviour improves without repeatedly supplying the entire past.

Remembering everything is not the goal. The goal is remembering the right information, at the right time, for the right user and task, in the smallest reliable form.

`memory_utility =`



A multi-objective system: every retained memory must pay for its latency, its tokens, its staleness risk and its privacy exposure.

IN THE LOOP Maximise the numerator, starve the denominator. That is the whole job.



Anatomy Of A Memory Unit

Overview of AI Memory

memory_unit

content	what information does it contain?
source	where did it come from?
owner	who owns it?
valid_at	when was it true?
confidence	how confident are we?
supersedes	what does it replace?
expires_at	when should it expire?
acl	which agents and users may retrieve it?
trace	can the answer reach its source?

NINE QUESTIONS, ONE RECORD

A unit that cannot answer these is a liability in the store, not an asset. Provenance, tenancy, time and confidence are fields, not afterthoughts.

This is stage 3 of the lifecycle made concrete: persistence with scope, provenance, timestamps, confidence, ownership and versioning.

IN THE LOOP Schema is the difference between a memory store and a pile of embeddings.



What A Memory Engineer Measures

Overview of AI Memory

RETRIEVAL QUALITY

- recall, MRR and nDCG on known-item sets
- citation and provenance coverage
- context tokens selected versus discarded

ANSWER QUALITY

- correctness on memory-dependent tasks
- faithfulness to what was actually stored
- grounded answers traceable to source

STATE HEALTH

- temporal and contradiction correctness
- stale-memory and cross-tenant leakage rates
- forgetting precision: remove the right things, keep the rest

ECONOMICS & OPERATIONS

- cache hit, bypass, invalidation and freshness rates
- latency across ingest, retrieve, rerank, generate
- cost per successful task; compaction fidelity
- workflow-promotion quality and regression rate

IN THE LOOP If it is not on a dashboard, it is folklore. These four panels are the memory dashboard.



Prompt Engineering	Context Engineering	Memory Engineering
Maximization— maximize input expressiveness Focuses on what and how much we feed into the LLM within a single forward pass. The aim is to saturate the model's reasoning potential through clear instructions, demonstrations, and framing. Front-loading cognition.	Optimization — optimize configuration and composition. Concerned with how contextual information is structured, ordered, compressed, and surfaced in the LLM's context window over time. The goal is efficient, high-signal usage of finite attention and token budget. Shaping working memory.	Continuation — preserve, consolidate, and evolve knowledge across time. Designs how experiences persist beyond the context window: what to store, how to retrieve, when to forget, and how to merge old knowledge with new reasoning. The aim is reliable, believable, and capable behavior that improves through use. Building long-term memory and metacognition.



Industry Update



Memory In The Wild: Five Labs, One Pattern

Industry Update · August 2026

LAB	CONSUMER MEMORY	BUILDER MEMORY	CONSOLIDATION	PORTABILITY	DISTINCTIVE ANGLE
OpenAI	ChatGPT memory; Dreaming V3 rolling out to every tier	None in the API; Codex keeps local Markdown memories	Dreaming background synthesis, roughly 5x cheaper to serve	No export path	Computer History turns desktop activity into memories and suggested skills
Anthropic	Claude memory on every tier since March 2026	API memory tool; Managed Agents memory; Claude Code MEMORY.md	Dreams async jobs plus Auto Dream upkeep; versioned, non-destructive	Imports ChatGPT and Gemini; full export	File-first memory plus a consolidation API for builders
Google	Gemini Memories; Personal Intelligence over Gmail, Photos, YouTube, Search	Memory Bank in Agent Engine; Gemini Enterprise workplace memory	Managed extraction; just-in-time selection against context packing	Memory import since April 2026	Memory over the whole Google data estate, with admin governance
Meta	Meta AI memory across Facebook, Messenger, WhatsApp and Instagram (1:1 chats)	No memory service; open weight-space research (memory layers, sparse finetuning)	Opaque profile fusion with social graph signals	Deletable, not portable	Memory fused with the social graph, and with the ads engine
Alibaba	Qwen App third-party agents built on reasoning and memory	AgentScope 1.0 with ReMe, fully open source	ReMe continuously indexes, links and consolidates Markdown memory files	Local-first files the user owns outright	Personal, task and tool memory levels; memory as linked files

IN THE LOOP Memory stopped being a feature and became a front: every frontier lab now ships it, and no two ship it the same way.



Same Instinct, Three Surfaces

Industry Update · August 2026

Within weeks of each other, both frontier labs reached for the sleep metaphor. AWS had already shipped the pattern as a managed service the year before.

OPENAI

Consumer scale

Dreaming V3 (June 2026) as the default memory for hundreds of millions: background synthesis replaces the note list, revises itself as time passes, and runs roughly 5x cheaper, opening the free tier.

ANTHROPIC

Developer primitive

Dreams (April to May 2026): consolidation as an async API job over one store plus up to 100 sessions, non-destructive and versioned; Auto Dream keeps Claude Code's MEMORY.md healthy.

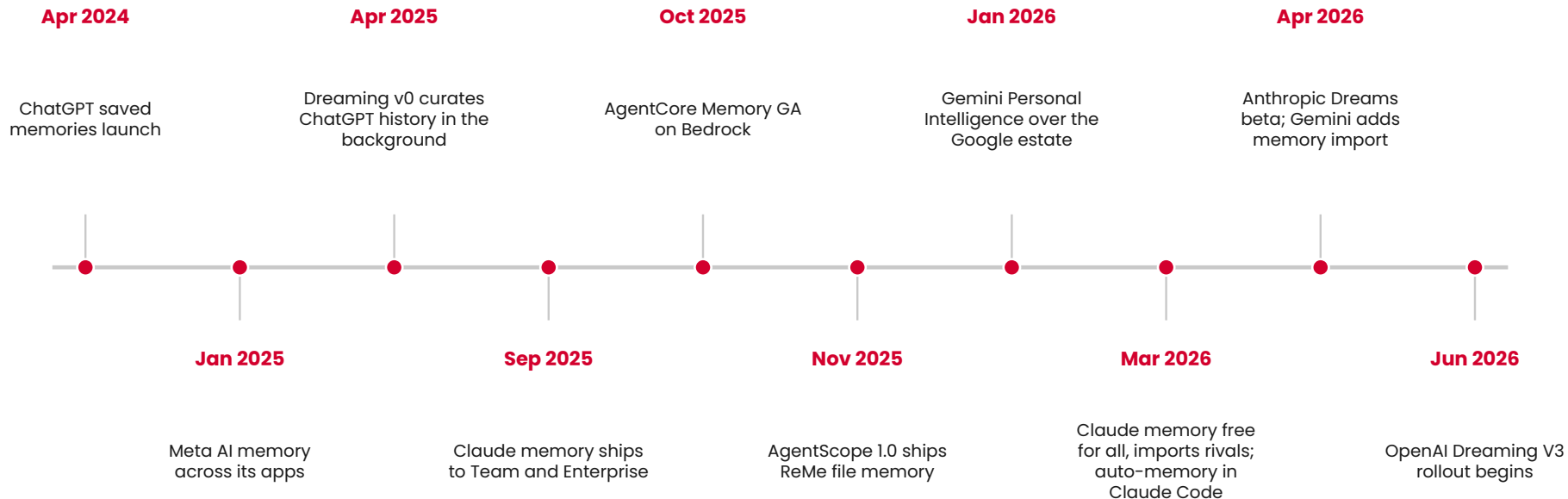
AWS AGENTCORE

Managed service

Long-term memory generated asynchronously after each write: raw context stored first, insights extracted and consolidated in the background, GA since October 2025.

From Saved Notes To Dreaming In Two Years

Industry Update · August 2026





What The Landscape Proves

Industry Update · August 2026

1 · DREAMING IS STANDARD

Background consolidation is now shipped practice at OpenAI, Anthropic and AWS, with two labs literally borrowing the sleep metaphor. Refine stopped being aspirational: it is the productised R.

→ *the key memory operation*

2 · MEMORY AS FILES

Codex memories, Claude Code's MEMORY.md, Alibaba's ReMe Markdown nodes and Letta's agent files arrived independently at the same substrate: memory as readable, editable files.

→ *the substrate conversation*

3 · PORTABILITY IS A FRONT

Anthropic and Google ship import tools; OpenAI still has no export path. Memory has become the moat, and moving it has become a competitive weapon.

→ *memory as the moat*

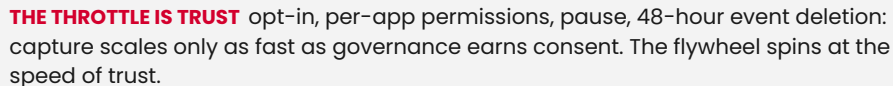
AND ONE FAILURE MODE, IN PUBLIC users documented Gemini's personal context being over-applied: saved details treated as high-priority instructions surfacing in every response regardless of relevance. Recall is not the hard part; **relevance gating and memory application** are.



AMBIENT CAPTURE, SHIPPED

FROM THE DOCS, VERBATIM FACTS

first-party warnings: files unencrypted on disk, and prompt-injection risk rises



Q&A





Break

Module 2:

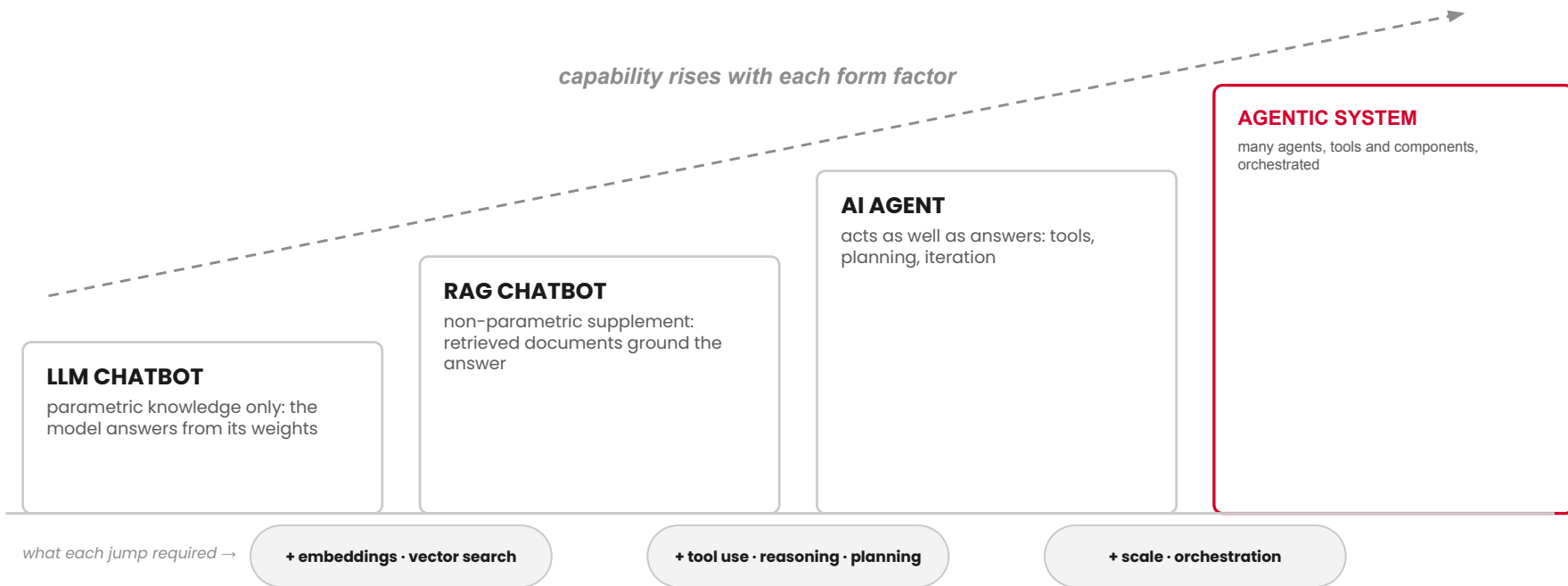
AI Memory Techniques Overview





Form Factors: How GenAI Applications Evolved

The Retrieval Layer



THE PATTERN every form factor adds a new way of reaching knowledge: weights, retrieval, tools, coordination. The next addition on this curve is **memory: the layer that makes what was retrieved and learned persist.**



Agent Memory Architecture: The Six Layers

The Layer Stack · map of this section

01

PROMPT

the instruction surface the model actually reads

chain of thought · ReAct · few-shot examples · instruction hierarchy

02

RETRIEVAL

how anything stored is ever found again

embedding models · lexical · vector · hybrid · reranking · quantization

03

CONTEXT

what enters the window, when, and at what budget

just-in-time retrieval · offloading · summarisation · compaction

04

MEMORY

what persists, improves and is allowed to fade

consolidation · augmentation · forgetting · substrate (files vs database)

05

SEMANTIC

what the terms in your data actually mean

comments & annotations · metric definitions · usage harvesting (v\$sql)

06

ONTOLOGY

what exists, and how it relates

entity types · typed relations · constraints · entity resolution

the model call reads from the top · meaning is governed at the bottom · every layer above depends on the ones below

IN THE LOOP Six layers, one architecture: the next slides take each layer's techniques in turn, with definitions, papers and reference architectures.



Chain Of Thought

The Prompt Layer · Techniques

PROMPT · 01

Chain of thought elicits **intermediate reasoning steps** before the answer: either a worked exemplar in the prompt, or the zero-shot trigger **“let’s think step by step”**. Accuracy on multi-step problems rises sharply at scale.

PAPERS & LINEAGE

Wei et al. 2022 *Chain-of-Thought Prompting Elicits Reasoning in LLMs*

Kojima et al. 2022 *LLMs are Zero-Shot Reasoners (“step by step”)*

Wang et al. 2022 *Self-Consistency: sample many chains, vote*

CONTEXT WINDOW · what goes in

SYSTEM
role · policies

QUESTION
“A juggler has 16 balls. Half are golf balls, half of those are blue. How many blue golf balls?”

THE CoT INVITATION
zero-shot: “let’s think step by step” · or one worked exemplar showing its reasoning



GENERATION · what comes out

“16 balls → 8 golf balls → half of 8 are blue → **4**”

the window carries only the invitation: the reasoning itself is output tokens, paid at generation time

IN THE LOOP A dozen tokens in the prompt buy hundreds of reasoning tokens in the output: chain of thought trades spend for accuracy.



ReAct: Reasoning + Acting

The Prompt Layer · Techniques

PROMPT · 02

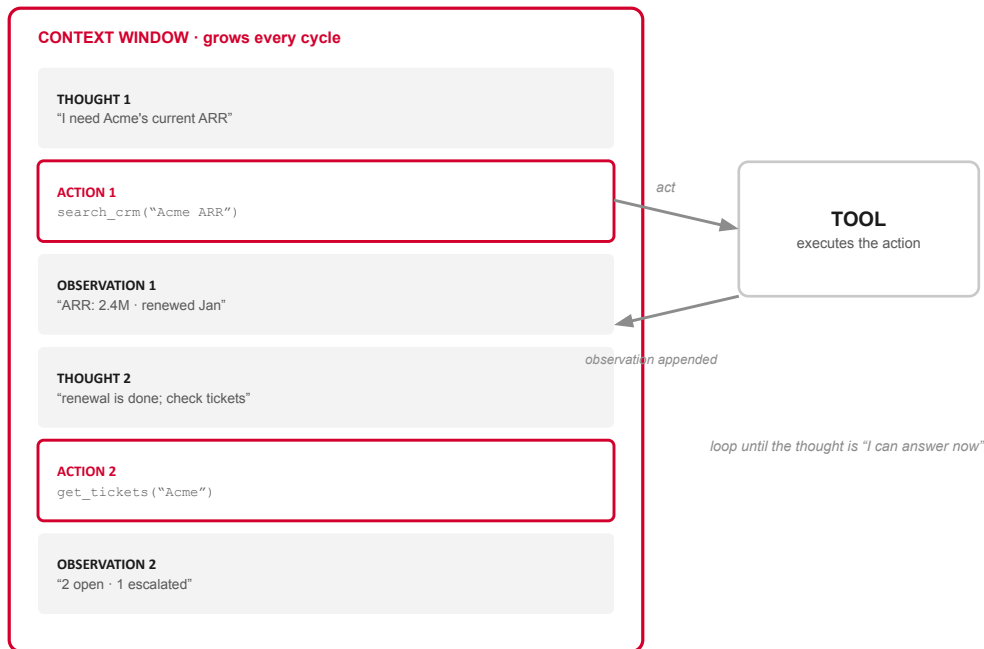
ReAct interleaves **thought, action and observation** in one loop: the model reasons about what it needs, **calls a tool**, reads the result, and reasons again. The pattern underneath nearly every modern agent harness.

PAPERS & LINEAGE

Yao et al. 2023 *ReAct: Synergizing Reasoning and Acting in LMs*

Schick et al. 2023 *Toolformer: models teach themselves tool use*

Shinn et al. 2023 *Reflexion: verbal self-feedback on failures*



each cycle appends ~3 blocks: ReAct windows grow fast, which is why compaction (context layer) exists

IN THE LOOP Thought, action, observation, repeat: the window becomes a transcript of the agent's working, one loop at a time.



Few-Shot Examples

The Prompt Layer · Techniques

PROMPT · 03

Few-shot prompting places **worked input-output examples** in the window so the model infers the mapping and applies it: **in-context learning**, with no weights changed. Format and label space matter as much as correctness.

PAPERS & LINEAGE

Brown et al. 2020 *Language Models are Few-Shot Learners (GPT-3)*

Min et al. 2022 *Rethinking Demonstrations: format over labels*

Liu et al. 2022 *What Makes Good Examples: retrieve similar ones*

CONTEXT WINDOW · the pattern is the prompt

SYSTEM

"classify support tickets by urgency"

EXAMPLE 1

"server down, revenue impact" → P1

EXAMPLE 2

"export button misaligned" → P4

EXAMPLE 3

"intermittent login failures" → P2

NEW INPUT

"checkout errors for EU users" → ?

→ P1

the model completes the pattern the examples establish

EXAMPLES ARE A BUDGET LINE

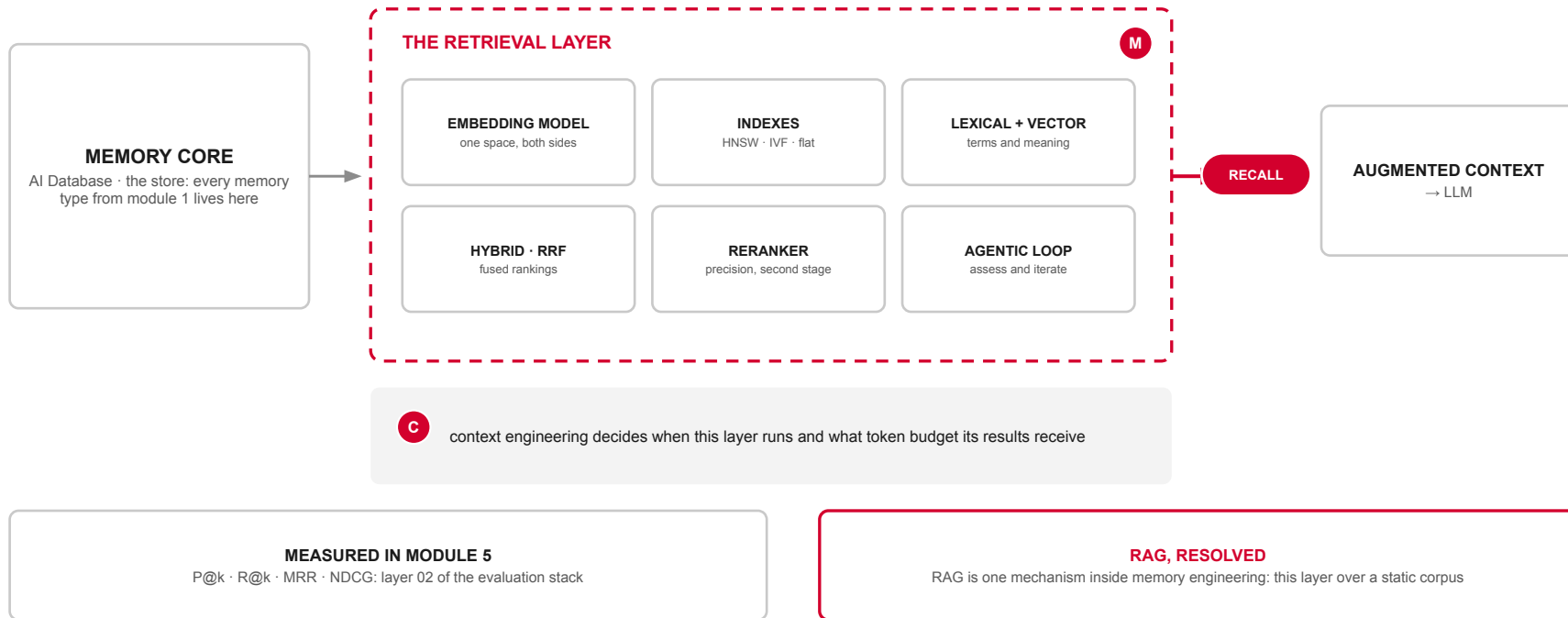
every shot costs tokens on every call: curate few, similar and well-formatted, or retrieve them per query

IN THE LOOP Three good examples teach the mapping; thirty mediocre ones teach the model to ramble and you to pay for it.



The Retrieval Layer, In Place

The Retrieval Layer



IN THE LOOP One layer between the store and the window: memory engineering builds it, context engineering invokes it, module 5 measures it.

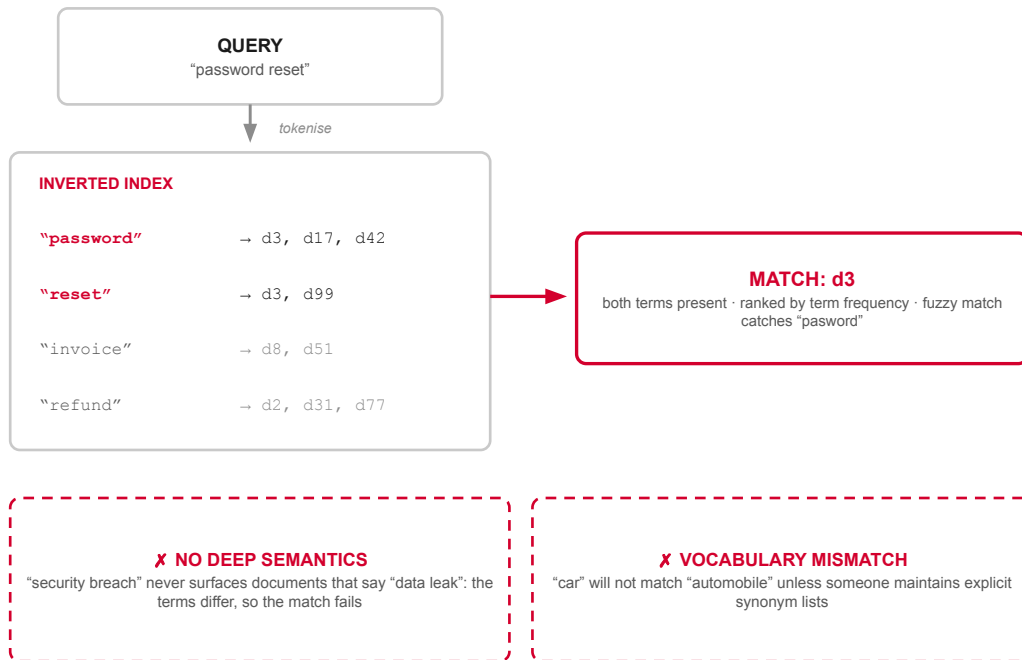


Lexical Search

The Retrieval Layer

METHOD 01 · TERMS

Lexical search matches **query tokens against an inverted index** of document tokens: term-based retrieval built on **keywords, term frequency, and exact or fuzzy string matching**. Fast, cheap, explainable, and blind to meaning.



these two gaps are exactly what vector search exists to close

IN THE LOOP Precision on the terms you typed, silence on the meaning you intended: lexical is half the answer, and the fast half.



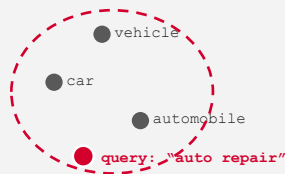
Embedding Models

The Retrieval Layer

THE ENCODER UNDER EVERYTHING

An embedding model is a **neural encoder** mapping inputs to **dense, fixed-length vectors** so that semantic similarity becomes **geometric closeness** under a metric. Trained **contrastively**: related items pulled together, unrelated pushed apart.

EMBEDDING SPACE · 2-D PROJECTION



nearest neighbours: the cluster the query lands in

WHAT IT POWERS

semantic search & RAG

clustering & dedup

recommendations

classification features

ONE SPACE, BOTH SIDES

queries and documents are encoded by the same model into the same space: that shared geometry is what makes vector search possible at all

IN THE LOOP The embedding model defines what similar means for your system; every retrieval decision downstream inherits its geometry.



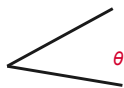
Distance Functions: What Similar Means

The Retrieval Layer

THE METRIC

A distance function turns intuition about similarity into **numbers**: it ranks candidates in nearest-neighbour search, and the metric you choose **defines what similar means** for your application.

COSINE



the angle between vectors: direction only, magnitude ignored

normalise embeddings first; the safe default for text

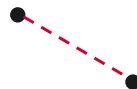
DOT PRODUCT



angle and magnitude together: longer vectors score higher

equivalent to cosine on normalised vectors; fast in practice

L2 · EUCLIDEAN



straight-line distance between the points themselves

natural for spaces where absolute position carries meaning

THE BINDING RULE

the metric must match how your embedding model was trained, and the index is built for one metric: changing it means rebuilding

IN THE LOOP Pick the metric with the model, not after it: similar is a definition you commit to at index-build time.

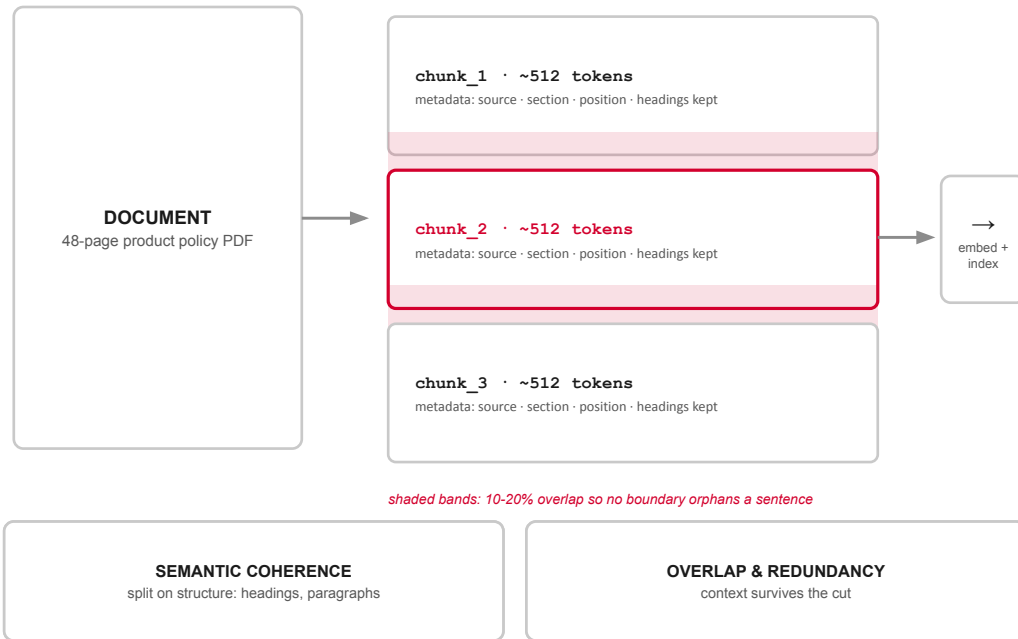


Chunking: Representation Before Retrieval

The Retrieval Layer

MEMORY-SIDE · STAGE 2 OF THE LIFECYCLE

Chunking breaks large documents into **smaller, retrievable pieces** before embedding or indexing. It decides the **granularity of everything downstream**: what a search can return is whatever a chunk was made to contain.



IN THE LOOP Retrieval can only return what representation prepared: chunk badly and no index, metric or reranker can save you.

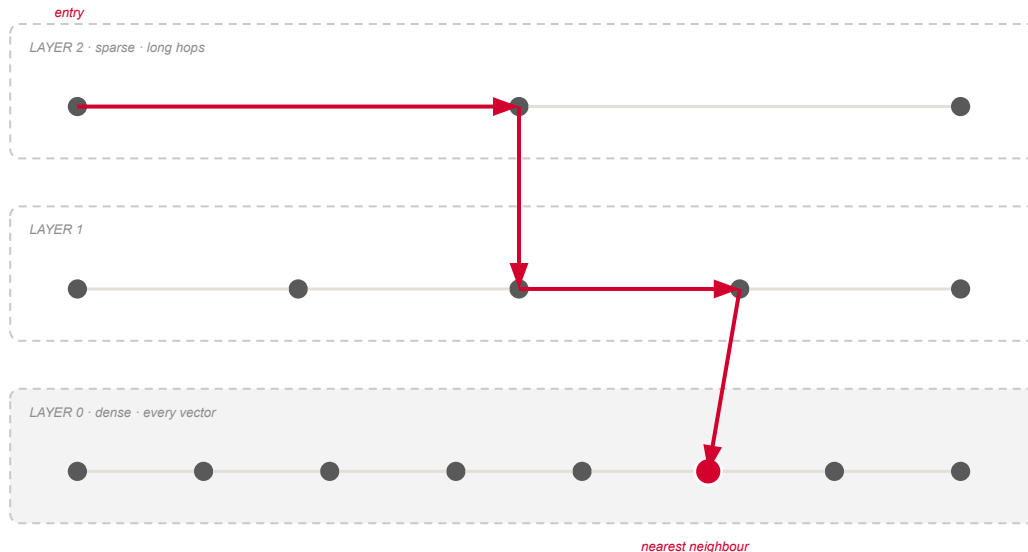


HNSW: The Index Behind Fast Vector Search

The Retrieval Layer

HIERARCHICAL NAVIGABLE SMALL WORLD

HNSW organises vectors into a **multi-layer small-world graph**. Queries enter a **sparse top layer**, move greedily toward closer nodes via long-range links, then **descend to denser layers** to refine: sub-linear search with high recall.



M neighbours per node: recall vs memory

efConstruction build breadth: graph quality vs build time

efSearch query breadth: recall vs latency

metric fixed at build: change = rebuild

IN THE LOOP Coarse at the top, exact at the bottom: a motorway network for vectors, with three dials trading recall, memory and latency.

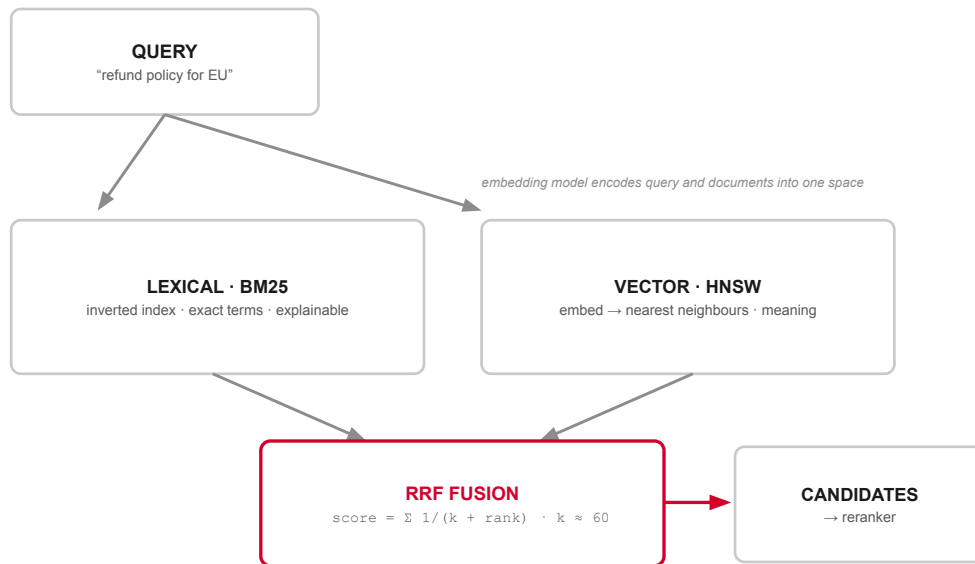


The Search Stack

The Retrieval Layer · Techniques

RETRIEVAL · 01

The workhorse pipeline: **lexical search** matches terms via inverted indexes (BM25), **vector search** matches meaning via embeddings and ANN indexes (HNSW), and **hybrid fusion** merges both rankings with RRF.



PAPERS & LINEAGE

Robertson & Zaragoza 2009 *BM25 and the probabilistic framework*

Karpukhin et al. 2020 *Dense Passage Retrieval (DPR)*

Malkov & Yashunin 2018 *HNSW graphs for ANN search*

Cormack et al. 2009 *Reciprocal Rank Fusion*

the full mechanism-by-mechanism treatment is module 2, The Retrieval Layer

IN THE LOOP Terms for precision, vectors for meaning, fusion for both: the stack every retrieval-augmented system stands on.



Reranking & Quantization

The Retrieval Layer · Techniques

RETRIEVAL · 02

Two levers on the quality-cost frontier: **cross-encoder reranking** buys precision by rescoring a shortlist with query and document read together, and **quantization** buys scale by compressing vectors 4x to 32x, searching coarse, then **rescoring at full precision**.

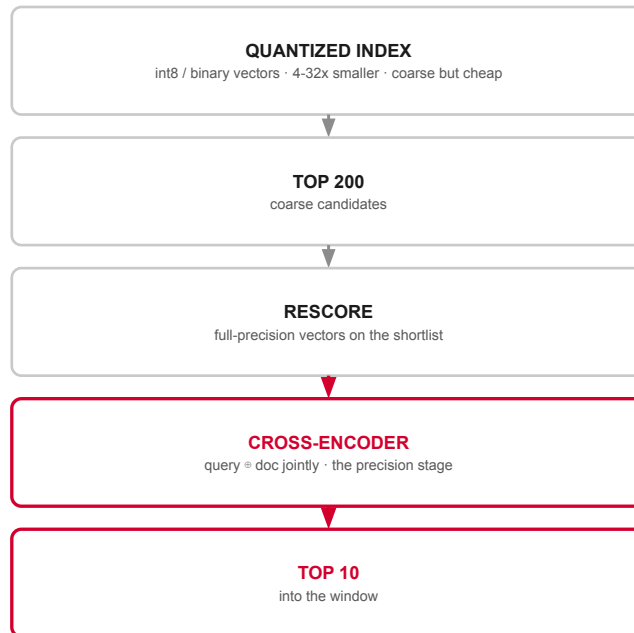
PAPERS & LINEAGE

Nogueira & Cho 2019 *Passage Re-ranking with BERT*

Khattab & Zaharia 2020 *ColBERT: late interaction*

Jégou et al. 2011 *Product Quantization for ANN*

Kusupati et al. 2022 *Matryoshka Representation Learning*



memory falls, recall barely moves: quantize the haystack, keep full precision for the needles

IN THE LOOP Compress where you search everything, spend where you judge almost nothing: the whole economics of retrieval in two moves.

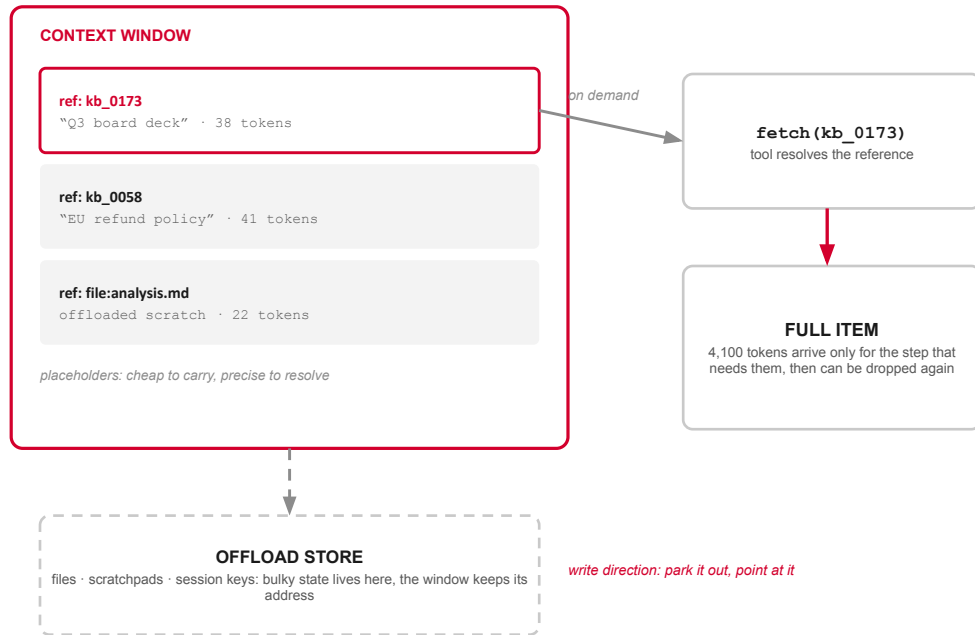


Just-In-Time Retrieval & Offloading

The Context Layer · Techniques

CONTEXT · 01

Keep **references, not contents**: the window holds a placeholder (an **ID plus a one-line gloss**), and a tool fetches the full item only when a step needs it. **Offloading** is the write direction: park bulky state outside, keep the pointer.



PAPERS & LINEAGE

Packer et al. 2023 *MemGPT: paging between window and storage*

Anthropic 2025 *Effective context engineering for AI agents*

Claude Skills 2025 *progressive disclosure: index now, body on use*

IN THE LOOP The window rents by the token; references pay studio prices for warehouse contents. Fetch on need, offload on bulk.



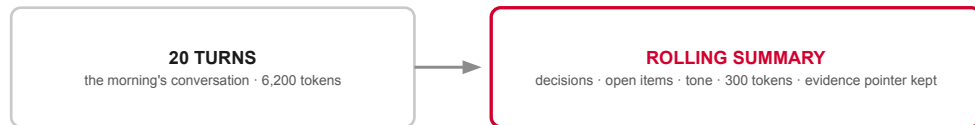
Context Reduction: Summarise & Compact

The Context Layer · Techniques

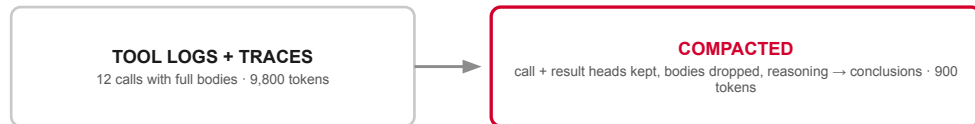
CONTEXT · 02

When the budget tightens, reduce: **summarisation** rewrites conversations and ageing window segments into short digests, and **compaction** strips tool logs and reasoning traces down to calls, results and conclusions. Keep a **pointer to the full record**.

LANE A · SUMMARISE (prose)



LANE B · COMPACT (structure)



PAPERS & LINEAGE

Wu et al. 2021 *Recursively Summarizing Books (OpenAI)*

Packer et al. 2023 *MemGPT: recursive summary on eviction*

Claude Code 2025 *auto-compact in production harnesses*



reduction is lossy by design: the pointer back to the full record is what makes the loss reversible

IN THE LOOP Prose gets summarised, structure gets compacted, and both leave a forwarding address: reduce the window, never the record.

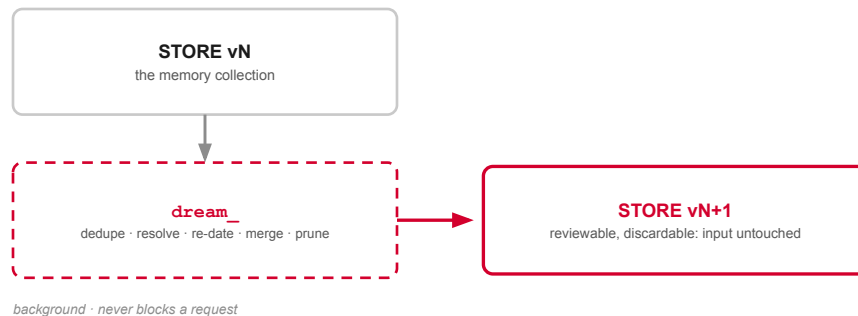


Consolidation & Forgetting

The Memory Layer · Techniques

MEMORY · 01

Consolidation is the **background pass over the store**: dedupe, resolve contradictions, absolute-date, merge and prune, producing **version N+1 without touching version N**. Forgetting is its gentler sibling: relevance **fades before anything is deleted**.



PAPERS & LINEAGE

Park et al. 2023 *Generative Agents: recency · importance · relevance*

Ebbinghaus 1885 *the forgetting curve: decay as design input*

OpenAI & Anthropic 2026 *Dreaming V3 and Dreams: shipped practice*

THE GENERATIVE AGENTS SCORE

$$\text{recall_score} = \alpha \cdot \text{recency} + \beta \cdot \text{importance} + \gamma \cdot \text{relevance}$$

recency decays exponentially: untouched, unimportant memories fade from recall long before any policy deletes them. Forgetting is a ranking outcome first, a deletion decision second.

forget-don't-delete: down-weight by default, delete only by explicit policy (retention, privacy, correction)

IN THE LOOP The store gets healthier while nobody is watching, and lets go the way people do: gradually, and only formally on purpose.

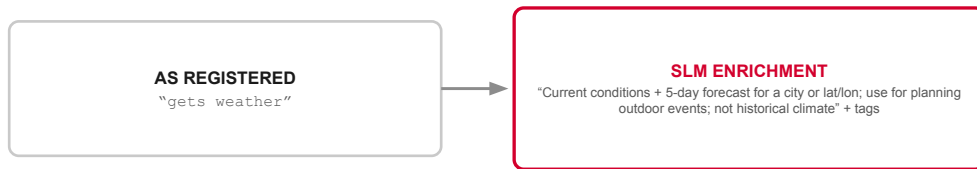


Augmentation & The Substrate Question

The Memory Layer · Techniques

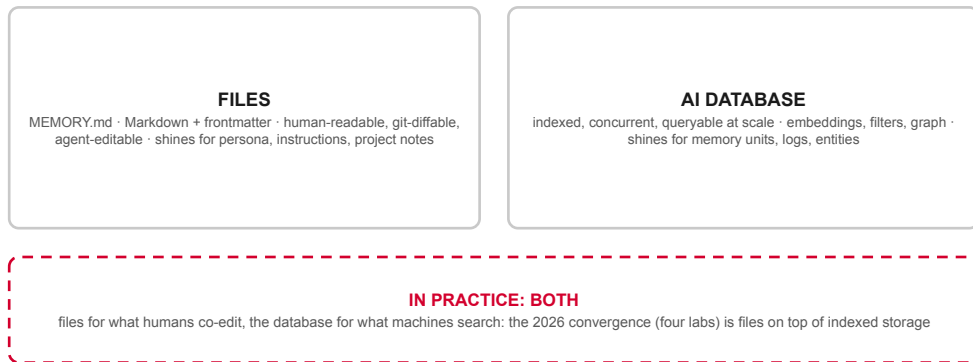
MEMORY · 02

Augmentation enriches memories **at ingestion**: a small model rewrites what is being stored (a tool's description, a unit's metadata) so it **retrieves better than it arrived**. The substrate question is where it all lives: **files or database**.



then embedded and stored: retrieval precision inherits the better description

THE SUBSTRATE QUESTION



PAPERS & LINEAGE

Gao et al. 2022 *HyDE: hypothetical documents as enrichment*

Industry 2025-26 *MEMORY.md · Codex memories · ReMe · Letta files*

IN THE LOOP Store things better than they arrived, and store them where their readers live: humans in files, retrieval in the database.



The Semantic Layer: Teaching Data Its Meaning

The Semantic Layer · Techniques

SEMANTIC · GOVERNED MEANING

The semantic layer carries **governed definitions**: what “revenue”, “active customer” or a cryptic column actually mean, written where the **agent can read them**. Without it, NL-to-SQL guesses; with it, generation is grounded.

PAPERS & LINEAGE

Yu et al. 2018 *Spider: schema understanding drives NL2SQL*

Li et al. 2023 *BIRD: real schemas, external knowledge needed*

COMMENTS & ANNOTATIONS

COMMENT ON COLUMN orders.stat_cd IS 'order status: C=complete, X=cancelled': inline meaning the agent retrieves with the schema

METRIC & VIEW DEFINITIONS

one governed definition of “active customer” as a semantic view: every generated query joins the same truth

USAGE HARVESTING · v\$sql

mine the queries experts actually run (v\$sql and friends): real join paths, filters and hints become few-shot examples for generation

WITHOUT → **WITH** “revenue by region”: plausible joins and a guessed metric become the governed view, the right join, the agreed number.

IN THE LOOP Semantic memory stores what the agent knows; the semantic layer governs what the words in your data mean. Same adjective, different job.



Ontology: What Exists, And How It Relates

The Ontology Layer · Techniques

ONTOLOGY · THE FOUNDATION

An ontology is a **formal specification of the entity types, relations and constraints** of a domain: not the data, but the **shape data must fit**. A taxonomy only ranks categories; an ontology says how things **connect and what is allowed**.

PAPERS & LINEAGE

Gruber 1993 *"explicit specification of a conceptualization"*

W3C 2004 *RDF & OWL: the web's ontology standards*

Google 2012 *Knowledge Graph: "things, not strings"*



CONSTRAINTS · what is allowed

an ORDER has exactly one CUSTOMER

a TICKET is raised by a CUSTOMER about a PRODUCT or ORDER

PRODUCT prices are attributes, not entities

IN THE LOOP Schema says what a table holds; taxonomy says what goes under what; ontology says what exists, how it relates, and what would be nonsense.



Ontology At Work In The Memory Stack

The Ontology Layer · Techniques

ONTOLOGY · WHAT IT BUYS AGENTS

In the memory stack the ontology does four jobs: **entity resolution** (many mentions, one thing), **typed relations** (multi-hop recall), **write validation** (contradictions caught at the door), and grounding the semantic layer's terms.

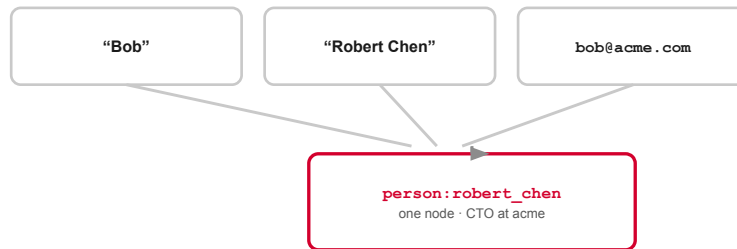
PAPERS & LINEAGE

Edge et al. 2024 *GraphRAG: community summaries over graphs*

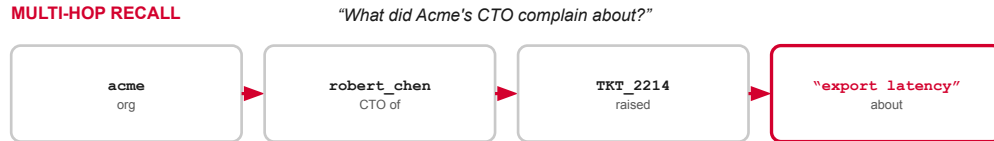
Zep / Graphiti 2025 *temporal knowledge graphs in production*

Park et al. 2023 *entity-centric memory in Generative Agents*

ENTITY RESOLUTION



MULTI-HOP RECALL



three typed hops the retrieval layer can traverse because the ontology typed them

WRITE VALIDATION

"order O-77 belongs to two customers" violates a constraint: the contradiction is caught at write time, not discovered at answer time

IN THE LOOP The ontology is why "Bob", "Robert" and an email address are one memory, and why the graph can walk from a company to a complaint.

Q&A





Break

Module 3:

Memory Engineering In Action





Memory Engineering As A Field

Overview of AI Memory

The discipline of designing the systems that let agents retain, recall, reuse and refine experience: the schemas, lifecycles, retrieval paths and consolidation loops that turn transient context into persistent, adaptive knowledge.

SITS BETWEEN

application engineering ⇄ data engineering ⇄ agent engineering ⇄ search optimization

THE AI MEMORY ENGINEER · ROLES & RESPONSIBILITIES

OWN THE SCHEMA

memory unit modelling, ontologies, entity resolution

OWN THE LIFECYCLE

generation to forgetting; forget, do not delete

OWN RECALL

retrieval, reranking, context assembly budgets

OWN THE EVOLUTION

consolidation pipelines and versioned promotion

OWN THE EVIDENCE

memory evals: freshness, continuity, relevance

OWN THE TRUST

provenance, PII, and the poisoning gate



What Is Memory Engineering?

Overview of AI Memory

memory engineering /'mem.ər.i en.dʒɪ'nɪə.rɪŋ/ noun

from the Latin **memoria**: mindful, remembering

- 1 Memory engineering is the discipline of designing the **cognitive and technical scaffolding** that transforms **transient context into persistent, adaptive knowledge**, governing how information flows between stores, the window, and back, and forming the **foundation for continual learning**.
- 2 *(in this course)* **State under policy**: the eleven-stage lifecycle, owned end to end.

memory = context, made durable

RETAINS

the write path, gated on entry

REFINES

consolidation and dreaming

GOVERNS

provenance, access, forgetting

LEARNS

token space and weight space



Memory Engineering: The Techniques

Overview of AI Memory

MEMORY UNIT MODELLING

typed records shaped by the ontology

EXTRACTION & PROMOTION

decide what deserves to persist

MEMORY TYPE IMPLEMENTATION

persona · toolbox · workflow · knowledge

INDEXING & RETRIEVAL

vector, graph and rerank paths

LIFECYCLE MANAGEMENT

TTL, decay, forget do not delete

CONSOLIDATION & DREAMING

background refine, versioned stores

SEMANTIC CACHE

answered once, reused cheaply

TEMPORAL MANAGEMENT

re-dating, validity windows, graphs

MEMORY EVALUATION

LoCoMo, LongMemEval, promotion gates

MEMORY SECURITY

provenance and the poisoning gate

CONTINUAL LEARNING · TOKEN SPACE

traces promoted into skills

CONTINUAL LEARNING · WEIGHT SPACE

traces distilled into fine-tunes

IN THE LOOP The first ten keep memory healthy; the last two turn it into learning. That is where module 06 hands over.

Resources

1. [“LangMem SDK for agent long-term memory”](#) (blog post, Feb 18, 2025) – Introduces the launch of the LangMem SDK, explaining its purpose, integration options, memory types, and how it enhances agent intelligence.
2. Developer conceptual guide: [“Long-term Memory in LLM Applications”](#) – Defines memory types (semantic, episodic, procedural) and how LangMem models them.
3. DeepLearning.AI short course: [“Long-Term Agentic Memory with LangGraph”](#) – A free, instructor-led video course (by Harrison Chase) that demonstrates constructing an agent with semantic, episodic, and procedural memory (e.g., email agent example with code) over several lessons.
4. ArXiv Research Paper: [“Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory”](#) (Apr 2025) – A formal research paper introducing Mem0 and its graph-based memory architecture. It outperforms existing memory methods on reasoning benchmarks (like LOCOMO), offering efficiency and improved performance.
5. Research Paper: [“MemGPT: Towards LLMs as Operating Systems”](#) (arXiv:2310.08560) – Introduces “virtual context management” to overcome LLM context limits, inspired by operating system memory hierarchies. Demonstrates improvements in document analysis and multi-session chat.
6. [Vertex AI Memory Bank](#) (Public Preview by Google) – Managed service allowing agents to extract, store, and retrieve memories across sessions using Gemini and semantic search. Integrated with Google’s Agent Development Kit (ADK) and compatible with frameworks like LangGraph and CrewAI

Q&A





Break

Poll Question

Question: After this course, where do you see yourself applying memory engineering techniques first?

- A) Customer support bots
- B) Research assistants / knowledge workers
- C) Multi-agent systems
- D) Other AI applications



Q&A





Break

Resources

1. [Anthropic – Multi-Agent Research System](#): Explains how complex, production-ready AI agents manage information. Notably avoids the term “*database*”, framing storage as “*memory*”—a shift in language important for database providers.
2. [Don't Just Build Agents, Build Memory-Augmented Agents](#) – Richmond Alake: Makes the case for memory-augmented architectures as the default for agent development. Analyzes Cognition and Anthropic examples; introduces AI memory engineering as key to reliability.
3. [Stanford CS229 – Building Large Language Models \(LLMs\)](#): A clear, technically solid overview of LLM architectures and the GenAI space. Balances theory and application—ideal foundation for understanding agent capabilities.
4. Survey: [Memory Mechanisms in LLM Agents](#): Detailed academic review of how memory is implemented in LLM-based agents. Covers architectures, techniques, and operational patterns.
5. Survey: [From Human Memory to AI Memory](#): Draws parallels between human cognitive memory and AI memory designs. Offers conceptual and technical framing for building more human-like agent memory.

The image features the O'Reilly logo in white, centered on a blue gradient background. The logo consists of the word "O'REILLY" in a bold, sans-serif font, followed by a registered trademark symbol (®). To the left of the text, there are two large, overlapping, semi-transparent blue circles that create a sense of depth and movement.

O'REILLY®